

SFAAO 2.4.0

Forensic Offline Signature Analysis Software

Technical, Scientific and Operational Manual

Consolidated Edition of Release 2.4.0

Diógenes Antonio Marques José

Mato Grosso State University - UNEMAT

SFAAO 2.4.0 - Multiplatform release documentation

INPI Registration BR512026005360-9

October 28, 2026

Technical-scientific and operational document. Software distribution without third-party datasets and without trained models.

Table of Contents

Release 2.4.0 documentation update **1**

1 Foundations, objectives and scope **3**

1.1 Overview . . . 3

1.1.1 Why SFAAO is important . . . 3

1.2 Platform philosophy . . . 5

1.3 Technical problem . . . 5

1.4 Objectives . . . 6

1.5 Scope and limitations . . . 6

1.6 Architectural principles . . . 6

1.6.1 Single responsibility . . . 6

1.6.2 Separation of responsibilities . . . 6

1.6.3 Low coupling . . . 6

1.6.4 High cohesion . . . 6

1.6.5 Modular architecture . . . 6

1.6.6 Reproducibility . . . 7

1.6.7 Traceability . . . 7

1.6.8 Controlled evolution . . . 7

1.7 Fundamental concepts . . . 7

2 Platform architecture and organization **11**

2.1 General architecture . . . 11

2.2 Project organization . . . 12

2.3 Graphical interface . . . 12

2.4 Controllers . . . 13

2.5 Persistence . . . 14

2.6 Storage . . . 14

3 Artificial intelligence core fundamentals **15**

3.1 AI core . . . 15

3.2 Modular AI framework . . . 15

3.3 Backbones . . . 16

3.4 Embeddings . . . 16

3.5 Loss functions . . . 17

3.5.1 Contrastive Loss . . . 17

3.5.2 Cosine Contrastive Loss . . . 17

3.5.3 Triplet Loss . . . 17
3.5.4 ArcFace . . . 17
3.6 Metrics and distances . . . 18
3.6.1 Cosine similarity . . . 18
3.6.2 Euclidean distance . . . 18
3.7 Preprocessing . . . 18
3.7.1 Compliance with the Official Architecture Specification . . . 19

4 Datasets, training and inference 20
4.1 Datasets and profiles . . . 20
4.2 Detailed dataset registration and validation pipeline . . . 20
4.3 PairBuilder . . . 21
4.3.1 Hyperparameters, configurable components, and training modes . . . 21
4.4 Detailed training pipeline . . . 27
4.5 ModelConfig . . . 28
4.6 TrainingConfig . . . 28
4.7 Early stopping [6] . . . 29
4.8 Model evaluation . . . 29
4.9 Detailed inference pipeline . . . 29

5 Forensic protocols, calibration and decisions 30
5.1 Forensic case . . . 30
5.2 Signature types: RC, RO, and QUESTIONED . . . 30
5.2.1 Human forensic classification of QUESTIONED signatures . . . 31
5.3 Statistical calibration . . . 31
5.4 Dual adaptive calibration . . . 32
5.5 Detailed FORENSIC_CASE pipeline — RC . . . 32
5.6 Detailed FORENSIC_CASE pipeline — RO . . . 32
5.7 Detailed SCIENTIFIC_EXTERNAL pipeline [16] . . . 33
5.7.1 RC scientific scenario . . . 34
5.7.2 RO scientific scenario . . . 34
5.7.3 Role of QUESTIONED signatures in the scientific protocol . . . 35
5.8 Scientific provenance [16] . . . 35
5.9 FAR, FRR, and EER . . . 36
5.9.1 FAR — False Acceptance Rate [5] . . . 36
5.9.2 FRR — False Rejection Rate [5] . . . 36
5.9.3 EER — Equal Error Rate [5] . . . 36
5.10 Thresholds, FAR target, and dataset shift [15], [16] . . . 37
5.10.1 Cosine similarity: higher values are more genuine . . . 37

5.10.2 Euclidean distance: lower values are more genuine . . . 38
5.11 Three-zone operational policy and threshold interval . . . 38
5.11.1 Aligned statistical domain: A1, A(N-1), and A(N) . . . 38
5.11.2 Thresholds and three-zone operational policy . . . 39
5.11.3 Conservative fold consolidation . . . 40
5.11.4 Calibration–inference alignment . . . 40
5.11.5 RC, RO, and scientific protocol semantics . . . 40
5.11.6 Fold persistence and auditable decision . . . 41
5.12 Detailed comparison pipeline . . . 41
5.13 Technical decision versus forensic classification [16] . . . 41

6 Traceability, auditing and security 42
6.1 Traceability . . . 42
6.2 Data hierarchy . . . 42
6.3 Model as a scientific artifact . . . 42
6.4 Auditing and integrity . . . 43
6.5 Deletion rules . . . 43
6.5.1 Data-access policy . . . 43
6.6 Security . . . 44
6.6.1 Authentication . . . 44
6.6.2 ADMIN . . . 44
6.6.3 EXAMINER . . . 44
6.6.4 Security and integrity bootstrap . . . 45
6.7 First administrator . . . 46
6.8 Storage security . . . 46

7 Reports and operational workflows 48
7.1 Detailed report-generation pipeline . . . 48
7.2 Complete operational flow . . . 48
7.2.1 Application startup pipeline . . . 51
7.2.2 Model-governance pipeline . . . 52
7.2.3 Dataset-governance pipeline . . . 52
7.2.4 Licensing, profiles, and quotas . . . 52
7.2.5 Startup and continuity security . . . 54

8 Requirements, installation and usage 55
8.1 Software dependencies, libraries, services, and requirements . . . 55
8.1.1 Python platform and runtime . . . 55
8.1.2 Main libraries . . . 55
8.1.3 Internet and pretrained weights . . . 56

8.1.4 Operating-system requirements . . . 56
8.1.5 Hardware . . . 56
8.1.6 Windows CPU installation . . . 56
8.1.7 Linux 2.4.0 CPU installation . . . 57
8.1.8 Protection of distributed code — PyArmor Pro . . . 57
8.2 Usage: adding a dataset . . . 58
8.3 Usage: models . . . 58
8.4 Usage: training . . . 59
8.5 Usage: calibration . . . 60
8.6 Usage: comparison . . . 62
8.6.1 Individual Comparison (LORO/A4) . . . 62
8.6.2 Inspect Pairwise Score . . . 63
8.6.3 Batch Comparison . . . 63
8.6.4 Refresh Context . . . 63
8.6.5 Threshold direction and interpretation . . . 63
8.7 Usage: report . . . 65

9 Governance, distribution and validation 68

9.1 Dataset policy and acquisition responsibility . . . 68
9.1.1 Datasets are not part of the commercial product . . . 68
9.1.2 Responsibility of the user or institution . . . 68
9.1.3 Support does not mean redistribution . . . 68
9.1.4 License status . . . 69
9.1.5 Dataset for training . . . 69
9.1.6 Dataset for SCIENTIFIC_EXTERNAL . . . 69
9.1.7 Institution-owned datasets . . . 69
9.1.8 Privacy, chain of custody, and rights . . . 70
9.2 Model policy . . . 70
9.3 Distribution . . . 70
9.3.1 What the buyer receives . . . 71
9.3.2 What the buyer does not receive . . . 71
9.3.3 Flow after acquisition . . . 71
9.4 Testing and validation . . . 71
9.4.1 Linux Mint, DEB, and Podman . . . 71
9.4.2 Windows 11 x64 CPU . . . 71
9.4.3 Red Hat (RPM): validated installation and operation . . . 72
9.4.4 Release evidence policy . . . 72
9.5 Limitations . . . 72

10 Best practices and detailed technical architecture **73**

10.1 Best practices . . . 73

10.2 Extensibility . . . 73

10.3 Recommended structure of the private repository . . . 73

10.3.1 Implementation map: modules, files, and classes . . . 74

10.3.2 Data model and SQLite schema . . . 76

10.3.3 Intellectual property and software identification . . . 79

10.3.4 Generative Artificial Intelligence Use Statement . . . 80

11 Glossary, checklists and closing remarks **81**

11.1 Glossary . . . 81

11.1.1 Glossary of training components . . . 82

11.2 Operational checklist . . . 82

11.3 Release checklist . . . 83

11.3.1 Validation status of the Manual and Release 2.4.0 . . . 83

11.4 Conclusion . . . 83

12 Bibliographic references and citation **85**

12.1 Scientific and technical references . . . 85

12.2 Relationship between manual concepts and references . . . 86

12.2.1 Recommended citations throughout the manual . . . 87

12.3 How to cite SFAAO . . . 87

12.3.1 Software reference . . . 87

12.3.2 Scientific reference to the methodology . . . 87

Release 2.4.0 documentation update — October 28, 2026

Software release, documentation, and scientific publication status

This manual preserves the architectural and scientific foundations of the previous edition and updates its operational procedures to the October 2026 SFAAO 2.4.0 artifacts. *Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification* has been **accepted for publication in IEEE Access**, according to the authors' confirmation. A final bibliographic citation (DOI, volume, pages, and publication date) will be supplied once publisher metadata is available. **Acceptance must not be misrepresented as already published.**

This revision records the Linux DEB/RPM and Windows CPU installers, the unified SHA-256 manifest, the DOCX report fix, the Windows launcher, persistent storage, and the four-option comparison interface. The 23 screenshots corresponding to the documented operational screens are included alongside the procedures. The earlier comparison-interface image is retained in the source-image archive only. Functional validation of release 2.4.0, including DOCX report generation, was confirmed by the technical owner on Windows 11, Linux Mint (DEB/Podman), and Red Hat (RPM). Cryptographic checks of all three installers complement, but do not replace, these functional tests.

Responsibility boundary

SFAAO produces computational technical evidence; it does not replace forensic handwriting expertise. The classification of QUESTIONED signatures and final handwriting examination conclusion remain the qualified examiner's responsibility.

Document control

Field	Value
Document	SFAAO 2.4.0 Technical, Scientific and Operational Manual
Manual version	2.4
Target software version	SFAAO 2.4.0
Nature	Technical-scientific and operational document
Status	Software validated on the reported platforms; screenshots included in this edition
Normative architectural source	`ARCHITECTURE_v2` — Official Platform Specification
Scientific-methodological source	Manuscript *Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification*
Distribution policy	Software without third-party datasets and without trained models
Target audience	Administrators, forensic examiners, researchers, technical teams, police institutions, and judicial institutions

0.0.0.1 Hierarchy of the sources used in this manual

This manual consolidates three levels of information:

1. **Normative architectural specification** — defines how the platform must be organized and evolve;
2. **Validated functional behavior** — describes workflows actually tested in release 2.4.0;
3. **Scientific foundation** — presents concepts and methods supported by the literature and by the SFAAO scientific manuscript.

When a statement represents an architectural rule, it must be understood as a normative platform requirement. When it represents validated behavior, it refers to the validation state documented for the current 2.4.0 cycle. When it represents a scientific foundation, the corresponding references are indicated in brackets.

> **Intellectual property.** While an intellectual-property protection procedure, patent application, NIT/AGINOV review, or institutional confidentiality guidance is in effect, this manual and the repository must remain private. A public version may be derived later.

Chapter 1

Foundations, objectives and scope

1.1 Overview

The **SFAAO — Forensic Offline Signature Analysis Software** is a modular computational platform designed to support forensic handwriting and signature examination. To make its purpose clear from the outset, the main terms used in this definition are presented below.

Core SFAAO technologies

Deep Learning is a form of Artificial Intelligence based on neural networks with multiple layers; it is used to learn features directly from images and is employed because offline signatures exhibit complex visual variation. A **Siamese neural network** uses branches with shared weights to produce comparable representations; it is used to learn proximity relationships between signatures. An **embedding** is the numerical vector produced to represent a signature; it is used to calculate similarity or distance. **Statistical calibration** estimates error rates and thresholds from comparison populations; it is used to adapt the decision to the model and protocol. **Traceability** is the ability to reconstruct which case, user, model, calibration, and configuration produced a given result; it is used because forensic results must be technically auditable.

SFAAO is not merely a classifier. It organizes the entire computational cycle required to register data, train models, calibrate thresholds, perform comparisons, preserve computational evidence, and produce auditable reports.

The system integrates:

- user management;
- case management;
- forensic examiner registration;
- dataset registration and validation;
- model training;
- model import, activation, and governance;
- embedding extraction;
- biometric comparison;
- PairBuilder;
- forensic calibration;
- external scientific calibration;
- FAR, FRR, and EER calculation;
- operational thresholds;
- traceability;
- auditing;
- referential integrity;
- DOCX/PDF reports;
- forensic image attachments.

1.1.1 Why SFAAO is important

Offline handwritten signature analysis works with an already produced image of the signature, such as a scan or photograph. Unlike a signature captured on a digitizing tablet, dynamic information such as stroke speed, pen pressure, acceleration, or temporal sequence is not directly available. For this reason,

offline verification is a complex computer-vision and biometric problem.

SFAAO was developed to add a quantitative, traceable, and reproducible computational layer to forensic handwriting and signature examination. The platform **does not replace the forensic handwriting examiner**. The professional remains responsible for the forensic analysis, for the human classification of questioned signatures when required by the protocol, and for the conclusion of the examination. SFAAO provides measurements, comparisons, statistics, traceability, and technical documentation that support this activity.

1.1.1.1 Human forensic examination and computational analysis are complementary

Criterion	Human handwriting examination	SFAAO / Deep Learning
Nature	Technical examination performed by a qualified professional, considering graphic characteristics, documentary context, and available material.	Mathematical processing of images by neural networks, embeddings, metrics, and statistical calibration.
Observed information	Graphic and documentary aspects interpreted in the context of the examination.	Visual features learned from image pixels.
Scalability	Limited by the time required for specialized analysis.	Allows a large number of comparisons to be automated after proper configuration.
Operational consistency	Depends on systematic application of the forensic method and documentation of the examination.	The same mathematical configuration is applied uniformly to the processed samples.
Reproducibility	Depends on documentation and examination conditions.	The same model, images, parameters, metric, and calibration make it possible to reconstruct the computational calculation.
Interpretation	Integrates technical elements and the broader forensic context.	Produces scores, distances, thresholds, and statistical rates that must be interpreted in the context of the case.
Final decision	Belongs to the professional responsible for the examination.	Does not issue an autonomous handwriting-examination conclusion; it provides complementary computational evidence.

1.1.1.2 Why use Deep Learning and CNNs

Deep Learning is an area of Artificial Intelligence that uses neural networks with multiple layers to learn representations from data. A **CNN — Convolutional Neural Network** is a neural network particularly suited to images because it learns local and hierarchical patterns such as edges, shapes, textures, and more complex combinations of these structures.

In SFAAO, CNNs and related architectures are used as feature extractors. Instead of relying only on manually designed measurements, the network learns numerical representations of signatures. These representations can be compared by similarity or distance.

This capability is important because signatures exhibit natural variability among genuine productions, differences in scale and acquisition, forgeries with different skill levels, and possible distribution shift between the dataset used for training and the real case.

1.1.1.3 Where SFAAO adds value

The platform is especially useful when forensic analysis must be combined with repeatable computational processing, for example in police and judicial institutions, questioned-document laboratories, audits, fraud prevention, financial institutions, document management, and scientific research in biometrics.

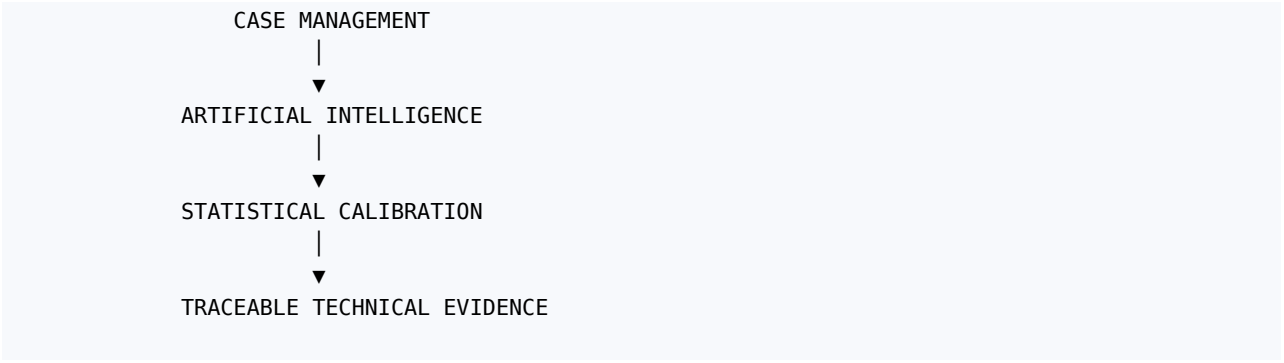
SFAAO's distinguishing feature is not merely the use of a neural network. It integrates, in a single workflow, case management, **RC** (signatures collected directly from the signatory while reproducing the pattern used in contracts), **RO** (signatures also collected directly from the signatory, but reproducing the pattern used in official documents), and **QUESTIONED** (all signatures present in the contract under examination), together with configurable training, forensic calibration, external scientific calibration, individual and batch comparison, traceability, integrity, licensing, and DOCX/PDF report generation.

Principle of complementarity

SFAAO must be understood as a technical-scientific support tool. The examiner interprets the material and assumes professional responsibility; the software performs computational processing in a controlled, recorded, and reproducible manner.

1.2 Platform philosophy

SFAAO must be understood as a computational forensic ecosystem:



AI produces representations and scores. Calibration interprets those scores statistically. The case organizes the forensic elements. Traceability makes it possible to reconstruct the path that produced each result.

1.3 Technical problem

Traditional biometric solutions may exhibit:

- fixed architecture;
- fixed backbone;
- dataset coupled to the code;
- global threshold;
- threshold obtained only during training;
- lack of adaptation to the case;
- lack of independent external calibration;
- limited traceability;
- limited reproducibility;
- lack of automated forensic documentation.

SFAAO addresses these limitations through modular architecture, dual calibration, case management, and complete persistence of technical parameters.

1.4 Objectives

- support the work of the forensic handwriting examiner;
- enable training of specialized models;
- support multiple backbones;
- support multiple loss functions;
- support multiple metrics;
- decouple datasets through profiles;
- enable case-specific calibration;
- enable independent scientific calibration;
- produce FAR, FRR, EER, and thresholds;
- generate auditable results;
- preserve history;
- produce technical documentation;
- enable reproducible scientific research;
- enable controlled architectural evolution.

1.5 Scope and limitations

SFAAO covers authentication, users, cases, signatures, datasets, models, training, evaluation, calibration, inference, comparison, reports, auditing, and storage.

The system does not replace the examiner. The computational result is quantitative support and must be interpreted within the full context of the examination.

1.6 Architectural principles

1.6.1 Single responsibility

Each class and module must have a clearly defined responsibility.

1.6.2 Separation of responsibilities

Training, inference, calibration, interface, persistence, security, and reports are independent components.

1.6.3 Low coupling

Modules communicate through public interfaces and well-defined structures.

1.6.4 High cohesion

Each module groups only functions that belong to the same domain.

1.6.5 Modular architecture

New features should preferably be incorporated through components with clear responsibilities. A **Factory** is a constructor that creates the correct object from the configuration; a **Repository** centralizes data access and persistence; a **Pipeline** is an organized sequence of processing stages; and a **Profile** describes structural rules for a resource, such as a dataset format. This separation reduces coupling and facilitates evolution, testing, and auditing.

1.6.6 Reproducibility

Configurations, seeds, metrics, thresholds, calibrations, artifacts, and platform version must be preserved.

1.6.7 Traceability

Every decision must be linkable to a user, case, model, calibration, execution, parameters, and version.

1.6.8 Controlled evolution

Structural changes should occur only when necessary.

1.7 Fundamental concepts

This section introduces essential terms assuming that the reader has no prior background in Artificial Intelligence, biometrics, statistics, or software development. When a concept appears again in later sections, it will be expanded only to the extent required to understand SFAAO, avoiding a full repetition of the same definition.

Artificial Intelligence — AI

What it is. An area of Computer Science that develops methods capable of performing tasks that require intelligent processing of information. **What it is used for in SFAAO.** To allow models to learn visual representations of signatures and produce quantitative measurements for comparison. **Why it is used.** Because an offline signature is a complex and variable image that is difficult to represent with a single manual rule.

Deep Learning

What it is. A subfield of AI based on neural networks with multiple layers. **What it is used for.** To automatically learn relevant features from images. **Why it is used in SFAAO.** It makes it possible to transform signature pixels into numerical representations suitable for biometric comparison.

CNN — Convolutional Neural Network

What it is. A type of neural network designed to process images through convolutional operations. **What it is used for.** To identify visual patterns at different levels of complexity. **Why it is used.** Backbones such as ResNet, EfficientNet, ConvNeXt, and MobileNet belong to the family of visual architectures used by SFAAO for feature extraction.

Offline signature

What it is. An image of a signature obtained after the act of writing has ended, such as a scan or photograph. **Important limitation.** The image does not directly contain writing speed, pressure, acceleration, or temporal sequence. **In SFAAO.** It is the type of biometric data analyzed by the platform.

Behavioral biometrics

What it is. Identification or verification based on features resulting from human behavior. **In the case of signatures.** The stroke is influenced by the writer's motor and graphic habits, although the offline version preserves only the visual result of that behavior.

Artificial Intelligence model

What it is. A mathematical structure with parameters learned from data. **What it is used for.** To receive an image and produce a representation or output useful for the task. **In SFAAO.** The model

is a versioned artifact associated with configuration, weights, metrics, and provenance.

Training and inference

Training is the stage in which the model adjusts its parameters using examples. **Inference** is the stage in which an already trained model processes new images without changing its weights. **In SFAAO.** Training creates or adjusts models; inference produces embeddings and scores used in calibration and comparison.

Dataset, sample, class, and label

A **dataset** is an organized collection of data. A **sample** is an item in that collection, for example a signature image. A **class** represents a category or identity. A **label** is the annotation associated with the sample or pair, indicating the supervised information used in training. **In SFAAO.** Datasets are accessed through profiles and are not redistributed with the platform.

Writer and signatory

In research datasets, **writer** is the identifier of the writer associated with the images. In a forensic case, **signatory** is the person to whom the signatures in the document are attributed and from whom the RC and RO reference standards are collected. The terms belong to different contexts and must not be confused.

Parameter and hyperparameter

A **parameter** is a value learned by the model during training, such as neural-network weights. A **hyperparameter** is a configuration chosen before or during training, such as learning rate, batch size, number of epochs, dropout, or scheduler. **What they are used for.** Hyperparameters control how training occurs and influence cost, stability, generalization, and reproducibility. **In SFAAO.** They must be persisted together with the model and training artifacts.

Overfitting, underfitting, and regularization

Overfitting occurs when the model adapts excessively to the training data and loses its ability to generalize. **Underfitting** occurs when the model does not learn the pattern sufficiently. **Regularization** encompasses techniques used to reduce overfitting, such as dropout and weight decay. These concepts help interpret hyperparameter choices and validation results.

Checkpoint

What it is. A file that stores model weights at a particular state. **What it is used for.** To load pretrained weights, resume or preserve training states, and package the best model. In SFAAO, checkpoints and weights must not be confused with the configuration that describes how the model was built.

Metric learning

What it is. A strategy in which the model learns a representation space where relationships between samples can be measured by distance or similarity. **What it is used for.** To make signatures from people comparable even when those people did not all need to be memorized as classes during training. PAIR and TRIPLET are modes related to this approach.

Backbone

What it is. A network that acts as the main feature extractor. **What it is used for.** To convert the image into a rich intermediate representation. **Why it is used.** It allows the visual architecture to be changed without rewriting the entire pipeline. SFAAO supports ResNet18, ResNet34, ResNet50,

EfficientNet-B0, ConvNeXt-Tiny, and MobileNetV3.

Embedding

What it is. A numerical vector that represents the signature in a mathematical space. **What it is used for.** To allow two signatures to be compared using a metric. **Why it is used.** It reduces a complex image to a representation suitable for calculating proximity or similarity.

Siamese network

What it is. An architecture with shared-weight branches that process different inputs using the same extractor [2]. **What it is used for.** To learn a space in which compatible examples tend to be closer and incompatible examples farther apart. **In SFAAO.** It is one of the metric-learning foundations used in comparison modes.

Loss — loss function

What it is. A mathematical function that measures the error that training attempts to reduce. **What it is used for.** To guide the optimizer in adjusting network parameters. **In SFAAO.** Contrastive, Cosine Contrastive, Triplet, and ArcFace are options compatible with specific training modes.

Optimizer and scheduler

An **optimizer** is the algorithm that uses gradients calculated by the loss function to update trainable parameters. A **scheduler** changes the learning rate over the course of training according to a rule. **In SFAAO.** Adam and AdamW are optimizers; none, StepLR, cosine, and plateau are scheduler options.

Score, similarity, and distance

A **score** is the numerical value obtained when comparing two representations. A **similarity** increases as representations become more alike; a **distance** tends to decrease as they become closer. **In SFAAO.** Cosine Similarity uses the rule “higher is more compatible”; Euclidean Distance uses “lower is more compatible”.

Threshold

What it is. A value used as a decision boundary. **What it is used for.** To convert a continuous score into decision regions. **Why it is important.** The same score may have a different interpretation depending on the model, metric, and calibration population; therefore, the threshold must be estimated and tracked.

Match, non-match, and impostor

A **match** is a comparison between samples treated by the protocol as belonging to the same identity. A **non-match** is a comparison between distinct identities or an impostor population. An **impostor** is a sample used to represent non-correspondence. An **external impostor** comes from an identity external to the signatory and, in the scientific protocol, from a legitimately obtained external dataset.

Calibration

What it is. A statistical process that uses match and non-match distributions to estimate error rates and thresholds. **What it is used for.** To adapt the decision to the defined model and context. **In SFAAO.** FORENSIC_CASE and SCIENTIFIC_EXTERNAL protocols are available.

FAR, FRR, and EER

FAR — False Acceptance Rate is the proportion of non-matches incorrectly accepted. **FRR — False Rejection Rate** is the proportion of matches incorrectly rejected. **EER — Equal Error Rate** is the rate associated with the point at which FAR and FRR approach each other. **What they are used for.** To quantify system behavior under a defined protocol and threshold.

RC, RO, and QUESTIONED

These three sets were defined in the first presentation of the platform. Section 31 formalizes their origin, forensic use, human classification of QUESTIONED signatures, and participation in calibration protocols.

Traceability and reproducibility

Traceability is the ability to reconstruct which user, case, model, calibration, configuration, and execution produced a result. **Reproducibility** is the ability to repeat processing under equivalent conditions. SFAAO preserves configurations, seeds, metrics, thresholds, UIDs, hashes, and artifacts to support these properties.

SHA-256 hash, integrity manifest, and ACL

A **SHA-256 hash** is a cryptographic digest of a file; changing the content changes the hash. An **integrity manifest** is a list of files and expected hashes. An **ACL — Access Control List** is the list of permissions that determines which accounts may access a given file or directory. **In SFAAO.** These mechanisms help detect code modification and restrict access to persistent storage.

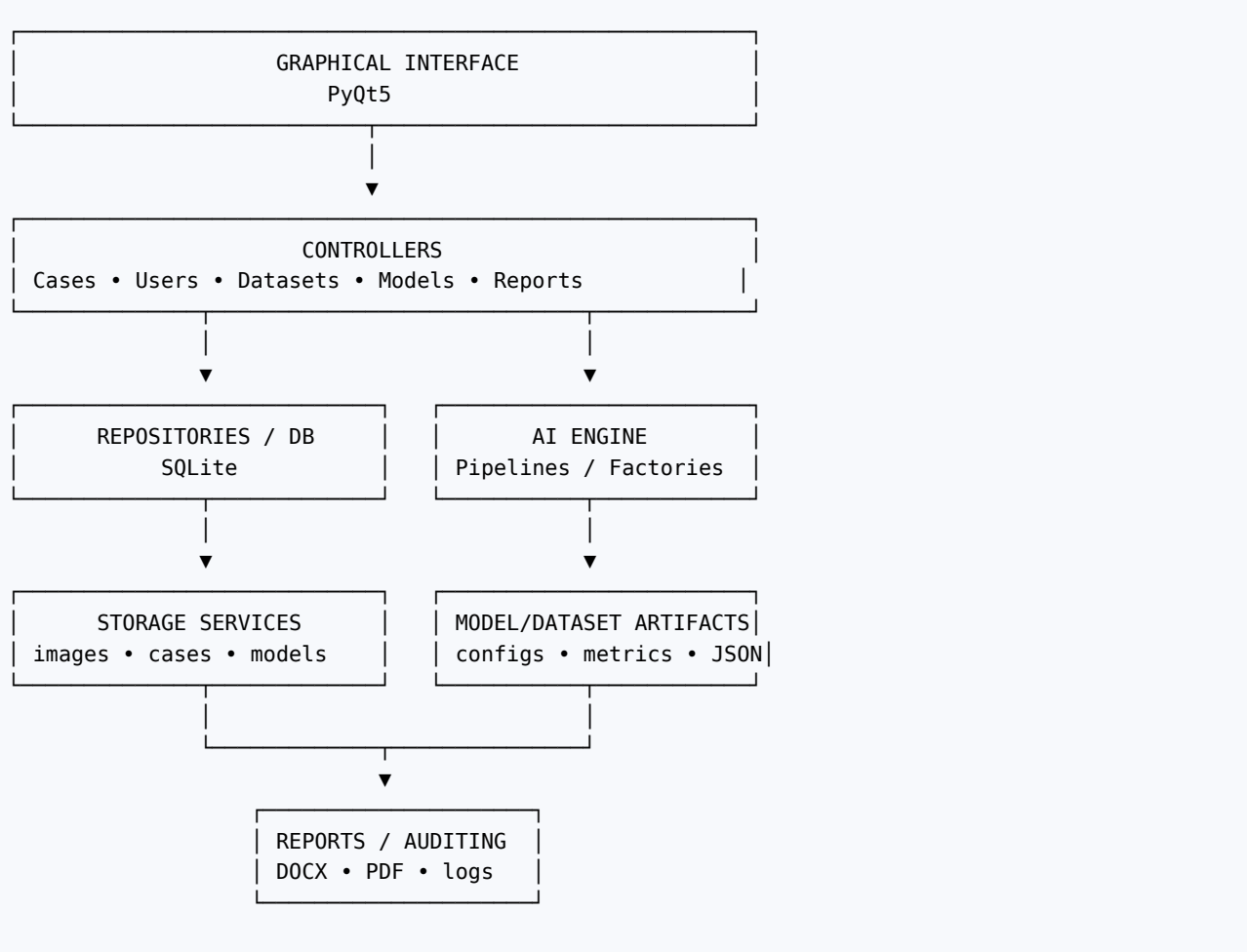
UID, fingerprint, and schema

A **UID — Unique Identifier** is an identifier created to stably distinguish an entity or execution, such as a case, calibration, or report. A **fingerprint** is a computational signature derived from data or context to identify a specific combination. A **schema** is the structure that defines database tables, fields, and relationships. These elements support traceability, consistency, and versioning.

Chapter 2

Platform architecture and organization

2.1 General architecture



How to read the architecture

The **GUI** is the graphical interface used by the person. A **Controller** coordinates the action requested by the GUI and calls the appropriate services. **SQLite** is an embedded relational database stored locally in files and used for metadata and history. The **filesystem** is the operating system's file and directory structure, used for images, models, and large artifacts. **Reports** are the components that transform persisted results into documents such as DOCX and PDF.

Consolidated release 2.4.0 architecture

Version 2.4.0 explicitly aligns training, calibration, inference, persistence, reporting, licensing, and integrity. The LORO decision domain is `A(N-1) / LORO_MEAN`; A1 remains a provenance component and A(N) remains descriptive. SQLite schema v6 records folds, alignment, and stability, while reports use canonical provenance fingerprint schema v3.

```
GUI / Controllers
  -> Training / Evaluation
```

```
  -> Calibration (AlignedScoreAggregator / LORO)
  -> Comparison (A(N-1) folds + conservative consolidation)
  -> SQLite schema v6 / questioned_decision_folds
  -> Reports / provenance fingerprint schema v3
  -> SecurityBootstrap / Licensing / Integrity
```

New runs must be `ALIGNED`; `MISMATCH` blocks execution and `LEGACY_MISMATCH` identifies historical records only.

2.2 Project organization

```
app.py
ai/
controllers/
database/
reports/
security/
storage_services/
ui/
assets/
scripts/
requirements.txt
```

- `ai/`: scientific algorithms;
- `controllers/`: orchestration;
- `database/`: schema, connection, and repositories;
- `reports/`: consolidation and report generation;
- `security/`: authentication and storage protection;
- `storage_services/`: physical storage;
- `ui/`: interface;
- `scripts/`: execution of auxiliary pipelines.

2.3 Graphical interface

The GUI is responsible for interaction, visualization, and parameter entry.

It **must not contain scientific logic**. The GUI may validate user input, display progress, and call controllers, but training, calibration, and decision calculations belong to the scientific core.

SFAAO 2.4.0 interface screenshots

The screens reproduced in this manual were supplied for the documentation of release 2.4.0. These examples illustrate system operation; demonstration data and illustrative charts must not be misrepresented as independent forensic or scientific results.

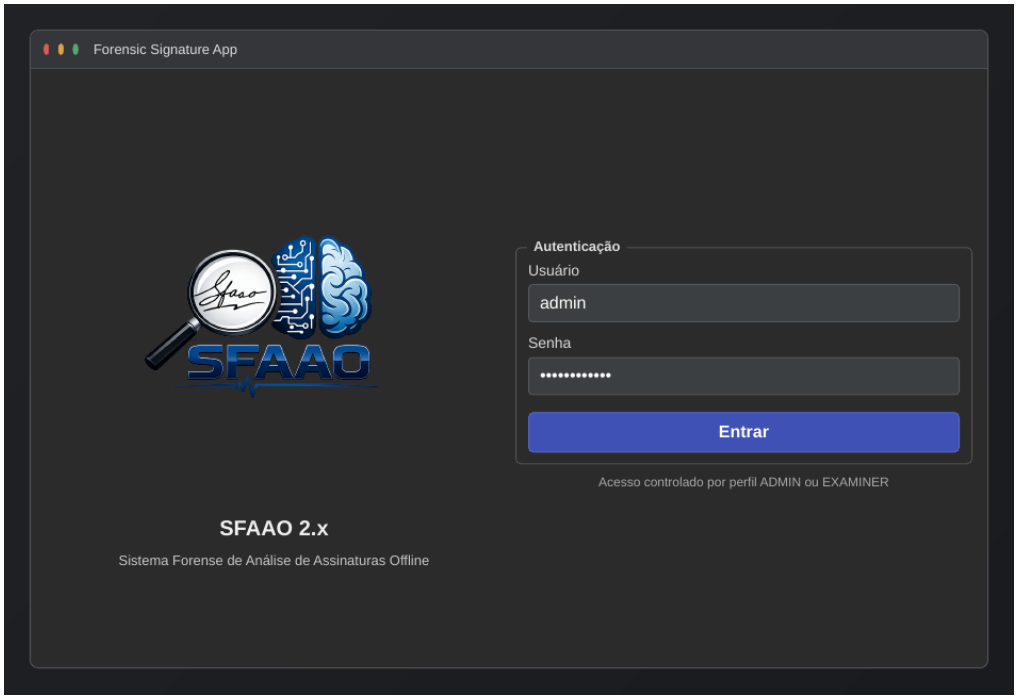


Figure 2.1: SFAAO 2.4.0 login window with application identity and authentication fields.

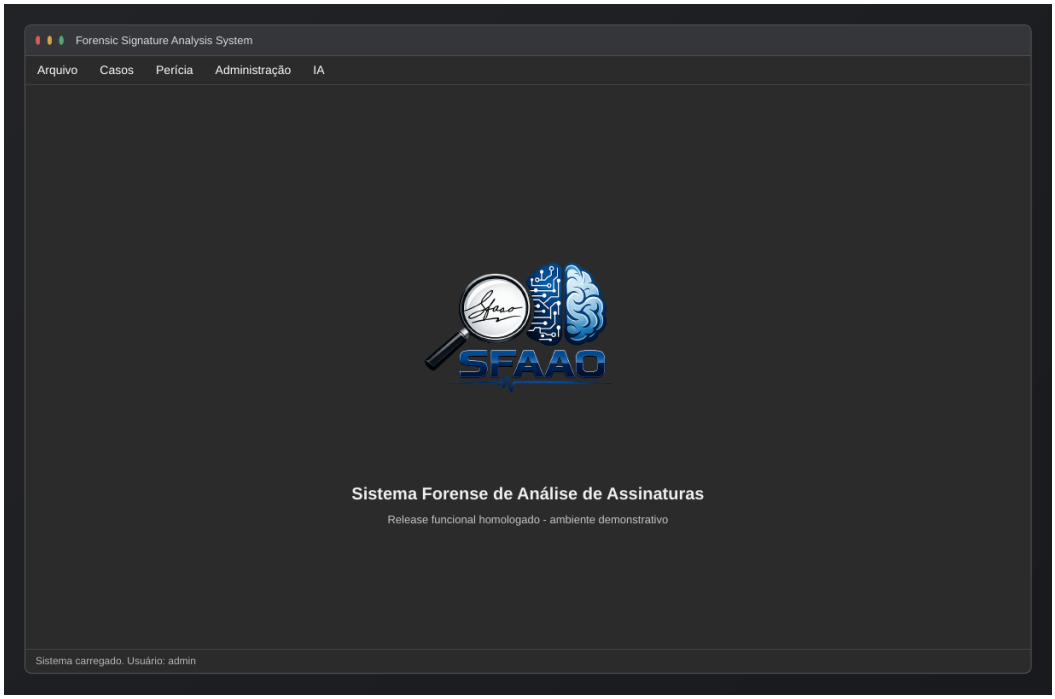


Figure 2.2: SFAAO main window under the administrator role, showing the platform menus.

2.4 Controllers

Typical flow:

```
GUI
 ↓
Controller
 ↓
```

```
Repository / Service / AI Pipeline
 ↓
Result
 ↓
GUI
```

Controllers coordinate operations but do not duplicate mathematical logic.

2.5 Persistence

SQLite stores structured metadata such as:

- users;
- cases;
- signatures;
- models;
- calibrations;
- comparison executions;
- results;
- reports;
- audits.

Neural weights, images, and large artifacts remain in the filesystem.

2.6 Storage

Storage contains:

- RC images;
- RO images;
- questioned signatures;
- models;
- calibrations;
- charts;
- results;
- reports.

Storage is organized by case and protected by filesystem permissions.

Chapter 3

Artificial intelligence core fundamentals

3.1 AI core

The `ai/` directory contains scientific logic and must not be aware of:

- authenticated user;
- role;
- permissions;
- widgets;
- GUI details.

It receives only parameters and paths previously authorized by the application.

3.2 Modular AI framework

The SFAAO AI framework is modular: the dataset is described by a profile, the user selects a **training mode**, and from that choice the system builds the appropriate sampling unit. **PAIR** learns from image pairs labeled as compatible or incompatible; it is used for metric learning with pair-based losses. **TRIPLET** learns from a reference (anchor), a sample from the same identity (positive), and one from a different identity (negative); it is used to enforce relative proximity relationships. **CLASSIFICATION** learns to distinguish known classes/identities in the dataset; it is used for conventional supervised training or compatible angular training. The general pipeline explicitly supports all three modes.

```
DATASET
  ↓
DATASET PROFILE
  ↓
TRAINING MODE
  ├─ PAIR           → positive/negative pairs
  ├─ TRIPLET        → anchor/positive/negative
  └─ CLASSIFICATION → image/class
  ↓
PREPROCESSING
  ↓
BACKBONE
  ↓
EMBEDDING / REPRESENTATION
  ↓
LOSS COMPATIBLE WITH THE MODE
  ↓
OPTIMIZER / SCHEDULER
  ↓
TRAINING
  ↓
VALIDATION
  ↓
EER / FAR / FRR / THRESHOLD
  ↓
MODEL PACKAGE
```

Compatibility rule

The sampling unit and the loss function must be compatible. In SFAAO, `PAIR` is intended for pair-based losses, `TRIPLET` for Triplet Loss, and `CLASSIFICATION` for compatible classification/angular losses. The GUI must not freely combine training mode and loss without validation by the AI core.

Factories decouple configuration from implementation:

```
BackboneFactory
ModelFactory
LossFactory
DistanceFactory
OptimizerFactory
SchedulerFactory
```

3.3 Backbones

Formal definition. The backbone is the network's feature extractor: it receives the preprocessed image and produces an intermediate representation that is projected into embedding space or sent to a classification head. In deep learning, its selection controls representational capacity, computational cost, memory use, and training/inference speed.

The ResNet family derives from the residual architecture of He et al. [4]. SFAAO 2.4.0 supports:

- **ResNet18** — compact residual architecture; a good option for baselines, quick tests, and lower-compute scenarios;
- **ResNet34** — intermediate residual architecture, deeper than ResNet18;
- **ResNet50** — deeper residual architecture based on bottleneck blocks and with higher computational cost;
- **EfficientNet-B0** — efficient architecture that balances depth, width, and resolution;
- **ConvNeXt-Tiny** — modern CNN whose design is inspired by recent architectural advances;
- **MobileNetV3** — lightweight architecture suited to scenarios in which efficiency and resource constraints are important.

Application in SFAAO. The backbone is selected in `ModelConfig` and built by `BackboneFactory`. Its output feeds the embedding layer/head corresponding to the training mode. The choice must be recorded as part of the model's identity and kept consistent across training, calibration, and inference.

Backbones validated in release 2.4.0

Real integration training runs were executed with **ResNet18, ResNet34, ResNet50, EfficientNet-B0, ConvNeXt-Tiny, and MobileNetV3**. These trials validate pipeline integration; they do not imply equivalent biometric performance across architectures.

When `pretrained=True`, the first run may require Internet access to obtain the checkpoint if it is not already in the PyTorch cache. Once stored locally, it can be used without a new download, subject to the installation dependencies.

3.4 Embeddings

An **embedding** is a numerical vector that summarizes signature features learned by the model. It serves as an intermediate representation so that two images can be compared mathematically. SFAAO uses

embeddings because directly comparing all pixels would be sensitive to acquisition variations and would not take advantage of the representation learned by the backbone.

The **embedding dimension** indicates how many numbers make up this vector and is a hyperparameter. Larger dimensions increase representational capacity and cost; smaller dimensions make the representation more compact. Configurable examples include:

```
64
128
256
512
1024
```

The dimension is part of the model's technical identity and must remain recorded.

3.5 Loss functions

The **loss function** numerically measures the error that training attempts to reduce. It is not the decision threshold and does not, by itself, represent forensic performance; its role is to guide the learning of the weights.

3.5.1 Contrastive Loss

What it is. A pair-based loss that pulls embeddings of positive pairs together and enforces separation for negative pairs [3]. **What it is used for.** To learn a geometry in which compatible samples are close. **Why use it.** It is a classical metric-learning formulation and integrates naturally with PAIR mode.

3.5.2 Cosine Contrastive Loss

What it is. A pair-based loss built on angular similarity. **What it is used for.** To penalize positives with insufficient similarity and negatives that are excessively similar. **Why use it.** It is appropriate when the relative orientation of embeddings is the primary signal for cosine comparison.

3.5.3 Triplet Loss

What it is. A loss that receives anchor, positive, and negative. **What it is used for.** To require the positive to remain closer to the anchor than the negative by a configured margin. **Why use it.** It learns relative relationships among three examples and is compatible with TRIPLET mode.

```
d(anchor, positive) + margin < d(anchor, negative)
```

3.5.4 ArcFace

What it is. An angular loss/head that adds a margin to the correct class in angular space. **What it is used for.** To increase separation between identities and compactness within the same class. **Why use it.** It can produce discriminative representations in CLASSIFICATION training, provided that the number of classes, head, and configuration are compatible.

Operational rule

Loss and training mode must be compatible. SFAAO validates this relationship before executing the pipeline. Margin, scale, and angular margin are hyperparameters explained in Section 23.1.

3.6 Metrics and distances

A **comparison metric** transforms two embeddings into a numerical value. In SFAAO, the decision rule depends on the direction of the metric; therefore, “higher” and “lower” cannot be treated in the same way.

3.6.1 Cosine similarity

What it is. A measure of angular alignment between two vectors.

$$\cos(a, b) = \frac{a^{\mathsf{T}} b}{\|a\|_2 \|b\|_2}.$$

What it is used for. To compare the relative direction of embeddings while reducing the influence of their magnitude. **Why use it.** It is common in learned representation spaces and was used in the scientific study associated with SFAAO [16]. **SFAAO rule.** Higher similarity indicates greater compatibility; the binary decision accepts when the score satisfies $s \geq \tau$.

3.6.2 Euclidean distance

What it is. The geometric distance between two vectors.

$$d_E(a, b) = \sqrt{\sum_i (a_i - b_i)^2}.$$

What it is used for. To measure how far apart embeddings are in the space. **Why use it.** It is a direct proximity metric and may be appropriate when the learned geometry preserves magnitude differences. **SFAAO rule.** Lower distance indicates greater compatibility; the final classification respects metric orientation and the three-zone operational policy defined by the calibrated thresholds.

Do not mechanically invert the metric

Cosine Similarity and Euclidean Distance are different metrics. Euclidean distance is not simply “cosine in reverse”. Thresholds, FAR, FRR, and comparison rules must respect the orientation recorded in the model and calibration.

3.7 Preprocessing

Preprocessing is the set of transformations applied to an image before it is sent to the model. It must be equivalent in training and inference; otherwise, the model receives data in a format different from the one on which it learned.

Typical operations include:

- **grayscale**: converts the image to shades of gray, reducing dependence on color when color is not required;
- **binarization**: separates pixels into two regions, usually stroke and background, when the protocol uses it;
- **aspect-ratio preservation**: avoids distorting the relationship between signature width and height;
- **padding**: adds margins to fit the image to the expected size without deforming it;
- **resize**: resizes the image to the backbone input resolution;
- **normalization**: transforms the statistical scale of pixels to the format expected by the model;
- **tensor**: multidimensional numerical structure used by PyTorch to execute neural-network operations.

A consolidated project configuration uses a 224×224 input.

3.7.1 Compliance with the Official Architecture Specification

The SFAAO 2.4.0 architecture is governed by the official `ARCHITECTURE_v2` specification. The following principles are normative:

```
GUI
 ↓
Case Management / Controllers
 ↓
AI Pipelines
 ├─ TrainingPipeline
 ├─ InferencePipeline
 └─ CalibrationPipeline
 ↓
Factories / Repositories / Configurations
 ↓
Networks / Datasets / Distances
 ↓
Persistence
```

The fundamental responsibilities are:

Layer	Responsibility
GUI	interaction, authentication, configuration, and visualization
Controllers	coordination of application operations
Database	metadata, history, auditing, and relationships
Storage	images, models, calibrations, charts, reports, and artifacts
AI	scientific algorithms with no knowledge of users, permissions, or widgets
Reports	consolidation and documentary materialization of results

3.7.1.1 Scientific isolation rule

The `ai/` module must not make authentication, authorization, or governance decisions. It receives parameters and paths already authorized by the application.

3.7.1.2 Case-model-calibration rule

Calibration belongs simultaneously to the **case** and the **model**. It must never be treated as a global parameter that can be reused indiscriminately.

3.7.1.3 Report rule

Every report must preserve, when applicable:

```
case
model
calibration
operational threshold
platform version
issue date
executions that produced the results
```

Chapter 4

Datasets, training and inference

4.1 Datasets and profiles

SFAAO separates the dataset **profile** from its **physical content**. Research associated with the system uses and discusses signature-verification databases such as GPDS, CEDAR, and BHSig260 [8]–[11], [14].

Profiles may be distributed. The data may not.

The profile abstracts:

- directory structure;
- file names;
- writer ID;
- genuine status;
- forgery status;
- splits;
- metadata.

This makes it possible to integrate new databases without changing the system core.

4.2 Detailed dataset registration and validation pipeline

```
ADMIN
 ↓
Add Dataset
 ↓
name + profile + version + directory
 ↓
DatasetReference
 ↓
DatasetProvider
 ↓
validate_profile_classification()
 ↓
scan images
 ├─ supported image?
 ├─ genuine?
 ├─ forgery?
 ├─ writer_id?
 ├─ genuine/forgery conflict?
 └─ unclassified?
 ↓
statistics
 ↓
final validation
 ↓
persistence in the SQLite catalog
 ↓
dataset available to the system
```

The scan has real 0–100% progress reporting.

4.3 PairBuilder

`PairBuilder` centralizes the construction of populations for training, validation, and calibration.

Possible types:

```
genuine × genuine
genuine × forgery
genuine × other writer
forgery × forgery
reference × external impostor
```

In Triplet mode:

```
anchor
positive
negative
```

Seeds must always be preserved whenever random sampling is used.

4.3.1 Hyperparameters, configurable components, and training modes

This section is the operational reference for values that the user may configure. The basic concepts of backbone, embedding, loss, optimizer, scheduler, score, and threshold were introduced in Section 7 and expanded in Sections 16–20; therefore, they are not fully redefined here. The focus is now on **what each configuration controls, what it is used for, why it is used, and what effect it has in SFAAO**.

4.3.1.1 Training modes

PAIR

What it is. A mode in which each training unit contains two images and a label indicating a positive or negative relationship. **What it is used for.** To teach the model to bring compatible pairs closer and separate incompatible pairs. **Why use it.** It is appropriate for metric learning with pair-based losses such as Contrastive and Cosine Contrastive.

```
Dataset
 ↓
PairBuilder
 ↓
positive/negative pairs
 ↓
image_1 + image_2
 ↓
shared backbone
 ↓
embedding_1 + embedding_2
 ↓
pair-based loss
 ↓
optimizer
```

TRIPLET

What it is. A mode in which each unit contains `anchor`, `positive`, and `negative`. The positive belongs to the same identity as the anchor; the negative belongs to a different identity. **What it is used for.** To make the positive remain closer to the anchor than the negative by a margin. **Why use it.** It allows direct learning of relative proximity relationships in embedding space.

```
Dataset
 ↓
Triplet Builder
 ↓
anchor + positive + negative
 ↓
shared backbone
 ↓
three embeddings
 ↓
Triplet Loss
 ↓
optimizer
```

CLASSIFICATION

What it is. A supervised mode in which each image has a class/identity and the model learns to separate those classes. **What it is used for.** To learn discriminative representations by identity. **Why use it.** It is appropriate when the dataset provides reliable classes and can be combined with classification heads or angular losses such as ArcFace when compatible.

```
image + class
 ↓
backbone
 ↓
representation
 ↓
classification / angular head
 ↓
compatible loss
 ↓
optimizer
```

Compatibility between mode and loss

PAIR is used with pair-based losses; TRIPLET with Triplet Loss; CLASSIFICATION with a compatible classification loss/head, including ArcFace when configured correctly. The platform validates combinations to prevent mathematically incompatible training from starting.

4.3.1.2 Architecture and representation hyperparameters

backbone_name

What it is. The name of the visual architecture used as the feature extractor. **What it is used for.** To select capacity, computational cost, and representation type. **Why it is configurable.** Different scenarios may require lighter or deeper models. **In SFAAO.** ResNet18, ResNet34, ResNet50, EfficientNet-B0, ConvNeXt-Tiny, and MobileNetV3 are validated integration options.

embedding_dim

What it is. Number of components in the embedding vector. **What it is used for.** Defines the size of the mathematical representation produced by the model. **Effect.** Larger dimensions may provide greater representational capacity but increase memory use and cost; smaller dimensions are more compact but may limit information. **In SFAAO.** The value is part of the model's technical identity and must be the same during loading, inference, calibration, and comparison.

dropout

What it is. A regularization technique that randomly disables a portion of activations during training. **What it is used for.** To reduce excessive dependence on specific units and help limit overfitting. **Effect.** A very high value may make learning difficult; a very low value provides little regularization. **In SFAAO.** It is persisted in ModelConfig.

pretrained

What it is. Indicates whether the backbone starts with weights previously learned on another dataset. **What it is used for.** To transfer generic visual knowledge and accelerate convergence. **Why use it.** It is useful with small or moderate datasets and in resource-constrained environments. **Effect.** `true` may require an initial checkpoint download when it is not already in the local cache; afterward, processing is local.

freeze_backbone

What it is. Indicates whether backbone weights remain frozen during training. **What it is used for.** To reduce cost, memory use, and overfitting risk when the goal is to train mainly the head/embedding. **Effect.** `false` enables fine-tuning of the extractor; `true` preserves its weights. **In SFAAO.** It must be interpreted together with pretrained and learning rate.

pretrained	freeze	Strategy	Interpretation
true	true	Frozen transfer learning	Faster and more economical; mainly trains the embedding/head.
true	false	Fine-tuning	Also adapts the backbone to the signature domain.
false	false	Training from scratch	Requires more data, epochs, and computational capacity.
false	true	Frozen initial weights	Generally not recommended if the weights are random.

4.3.1.3 Loss-function hyperparameters

margin

What it is. Separation margin used by contrastive or Triplet losses. **What it is used for.** To define how far incompatible examples should be separated. **Effect.** A margin that is too small may allow insufficient separation; one that is too large may make the constraint difficult to satisfy. **In SFAAO.** The exact meaning depends on the selected loss and must be persisted.

ArcFace: scale and angular margin

Scale (s) multiplies angular logits to control their numerical scale. **Angular margin (m)** increases the separation required between classes in angular space. **What they are used for.** To make identities more separable in compatible CLASSIFICATION training. **Effect.** Inappropriate values may cause instability or insufficient separation. Both are part of the model's reproducible configuration.

4.3.1.4 Optimization hyperparameters

optimizer_name

What it is. The choice of algorithm used to update the weights. **What it is used for.** To control convergence dynamics. **In SFAAO.** `adam` and `adamw` are supported. AdamW applies weight decay in a decoupled manner and may be preferable when this regularization is part of the experiment.

learning rate — lr

What it is. The basic size of weight updates. **What it is used for.** To balance training speed and stability. **Effect.** A value that is too high may cause instability or prevent convergence; a value that is too low may make training excessively slow. **In SFAAO.** It must be analyzed together with optimizer, scheduler, batch size, and backbone freezing.

weight decay

What it is. Regularization applied to weights during optimization. **What it is used for.** To help limit excessive parameter growth and overfitting. **Effect.** An excessive value may reduce model capacity too much; a zero value disables this form of regularization.

batch_size

What it is. Number of training units processed before an optimizer update. **What it is used for.** To control memory, throughput, and gradient variability. **In SFAAO.** A unit is a pair in PAIR, a triplet in TRIPLET, and an image/class in CLASSIFICATION. A larger batch consumes more memory.

epochs

What it is. Maximum number of planned training passes over the defined sampling. An **epoch** represents one training cycle under the pipeline. **What it is used for.** To control how long the model may learn. **Effect.** Too few epochs may cause underfitting; too many may increase cost and overfitting. Early stopping may terminate training before the maximum.

4.3.1.5 Learning-rate schedulers

scheduler_name

What it is. A rule that modifies the learning rate across epochs. **What it is used for.** To allow larger updates at the beginning and/or smaller ones as training matures. **In SFAAO.** `none` keeps the learning rate without a schedule; `StepLR` reduces it at intervals; `cosine` follows a cosine schedule; `plateau` reduces it when the monitored metric stops improving. Scheduler-specific parameters must also be recorded.

4.3.1.6 Sampling hyperparameters

train_pairs and val_pairs

What they are. Planned numbers of training and validation samples in pair mode, or equivalent size parameters in other modes. **What they are used for.** To control computational volume and the amount of evidence used for learning and evaluation. **Why keep them separate.** Validation must not be used for every weight update; it measures behavior during development and supports selection of the best state.

positive_ratio

What it is. Planned proportion of positive pairs. **What it is used for.** To control the balance between examples from the same identity and examples from different identities. **Effect.** A highly imbalanced set may bias learning. **In SFAAO.** Both the requested and produced distributions must be recorded.

forgery_negative_ratio

What it is. Planned fraction of negative samples formed by forgeries from the same writer/profile, when the dataset allows it. **What it is used for.** To control the proportion of potentially harder negatives relative to negatives from other writers. **Why it is used.** It allows controlled study of the model's ability to separate forgeries and different identities.

seed and determinism

A **seed** is the initial value of pseudorandom number generators. **What it is used for.** To make sampling of pairs, triplets, subsets, and impostors more repeatable. **Determinism** refers to the effort to reduce variation across runs. Using the same seed improves reproducibility but does not guarantee bit-for-bit identical results across different hardware, kernels, and library versions.

4.3.1.7 Early stopping

min_epochs

What it is. Minimum number of epochs that must be executed before early stopping is allowed. **What it is used for.** To avoid terminating training too early because of initial fluctuations.

patience

What it is. Number of consecutive evaluations tolerated without sufficient improvement. **What it is used for.** To give training a chance to recover improvement before stopping. A larger value makes early stopping more tolerant and may prolong training.

min_delta

What it is. Minimum improvement required to consider that the metric has actually improved. **What it is used for.** To prevent small numerical fluctuations from indefinitely resetting patience. **In SFAAO.** Early stopping may monitor EER or another metric defined by the pipeline.

```
execute at least min_epochs
 ↓
evaluate metric
 ↓
improvement >= min_delta?
 ├─ YES → save best state and reset patience
 └─ NO  → increment patience
 ↓
patience exhausted?
 ├─ YES → stop training
 └─ NO  → continue
```

4.3.1.8 Execution and hardware

device

What it is. Device on which PyTorch executes tensors and the model. **Typical values.** CPU, GPU/CUDA, or automatic selection. **What it is used for.** To control where computation takes place. **In the CPU 2.4.0 distribution.** The Linux container and Windows installer target CPU operation. The Windows 11 validation cycle included functional training; the actual execution device must be recorded for each experiment.

num_workers

What it is. Number of auxiliary processes used by the data loader. **What it is used for.** To parallelize reading and preprocessing when the environment permits. **Effect.** More workers may improve throughput, but they increase resource use and may cause incompatibilities in certain environments. For the Linux 2.4.0 validation experiment, the effective value must be recorded with the executed configuration; historical Windows evidence is not automatically promoted to this release.

4.3.1.9 Best state, metrics, and artifacts

Best state dict

What it is. A copy of the model weights at the best state found according to the validation criterion. **What it is used for.** To prevent the final epoch from replacing an earlier superior state. **In SFAAO.** The best state is restored at the end and the corresponding epoch is recorded.

During and after training, the following may be persisted, among other items:

```
epoch
train_loss
val_loss
EER
threshold
learning_rate
best_epoch
best_validation_loss
epochs_requested
epochs_executed
final_eer
final_threshold
history
```

Evaluation may generate FAR×FRR curves and score distributions. These artifacts are not merely illustrations: they document how the populations behaved and help audit the estimated threshold.

4.3.1.10 Summary matrix

Element	Options / examples in SFAAO
Training mode	PAIR, TRIPLET, CLASSIFICATION
Backbone	ResNet18, ResNet34, ResNet50, EfficientNet-B0, ConvNeXt-Tiny, MobileNetV3
Loss	Contrastive, Cosine Contrastive, Triplet, ArcFace
Comparison	Cosine Similarity, Euclidean Distance
Embedding	configurable dimension

Element	Options / examples in SFAAO
Regularization	dropout, weight decay
Initialization	pretrained true/false
Fine-tuning	freeze_backbone true/false
Optimizer	Adam, AdamW
Scheduler	none, StepLR, cosine, plateau
Sampling	train/validation, positive ratio, forgery negative ratio
Reproducibility	seed, determinism, history, and persisted configurations
Early stopping	min_epochs, patience, min_delta
Execution	device, num_workers

4.4 Detailed training pipeline

```
01. user selects dataset
02. selects backbone
03. selects loss
04. selects metric/distance
05. defines embedding_dim
06. defines dropout
07. defines pretrained
08. defines freeze_backbone
09. defines epochs
10. defines batch_size
11. defines learning rate
12. defines weight_decay
13. defines seed
14. defines number of samples
15. defines positive_ratio
16. defines forgery_negative_ratio
17. defines optimizer
18. defines scheduler
19. defines early stopping
20. ModelConfig is persisted
21. TrainingConfig is persisted
22. DatasetProvider loads dataset
23. builder generates pairs/triplets
24. training and validation are separated
25. DataLoaders are built
26. image is preprocessed
27. backbone extracts features
28. head produces embedding
29. loss is calculated
30. backward()
31. optimizer.step()
32. scheduler.step() when applicable
33. validation epoch
34. generation of match/non-match scores
35. threshold sweep
36. FAR calculation
37. FRR calculation
38. EER calculation
39. calculation of τ_EER
40. update best state
41. evaluate early stopping
42. restore best state
```

```
43. final evaluation
44. save weights
45. save configurations
46. save metrics
47. generate FAR×FRR
48. generate score distribution
49. create model package
50. automatic registration in Model Manager
```

4.5 ModelConfig

`ModelConfig` records the architectural identity of the model. The concepts below were already defined in Sections 7, 16–19, and 23.1; here the objective is only to show which values must remain associated with the artifact so that it can be loaded and reproduced correctly.

```
backbone_name
pretrained
freeze_backbone
embedding_dim
dropout
loss_name
distance_name
loss-specific parameters, when applicable
```

The configuration must accompany the model. Silently changing any of these values may make training, inference, calibration, and comparison incompatible.

4.6 TrainingConfig

`TrainingConfig` groups the hyperparameters that control training execution. Their definitions and effects are consolidated in Section 23.1.

```
epochs
batch_size
learning_rate
weight_decay
seed
train_pairs / training size
val_pairs / validation size
positive_ratio
forgery_negative_ratio
min_epochs
patience
min_delta
device
num_workers
optimizer_name
scheduler_name
deterministic
```

The persisted file must record the values actually used, not only the interface defaults.

4.7 Early stopping [6]

The early-stopping mechanism, along with `min_epochs`, `patience`, and `min_delta`, was defined in Section 23.1. At this stage of the pipeline, SFAAO monitors the configured metric, preserves the best state, and stops training when the stopping criterion is satisfied.

The objective is to avoid unnecessary computational cost and reduce the risk of continuing training after validation stops improving. The stopping decision, number of executed epochs, and best epoch must remain recorded in the artifacts.

4.8 Model evaluation

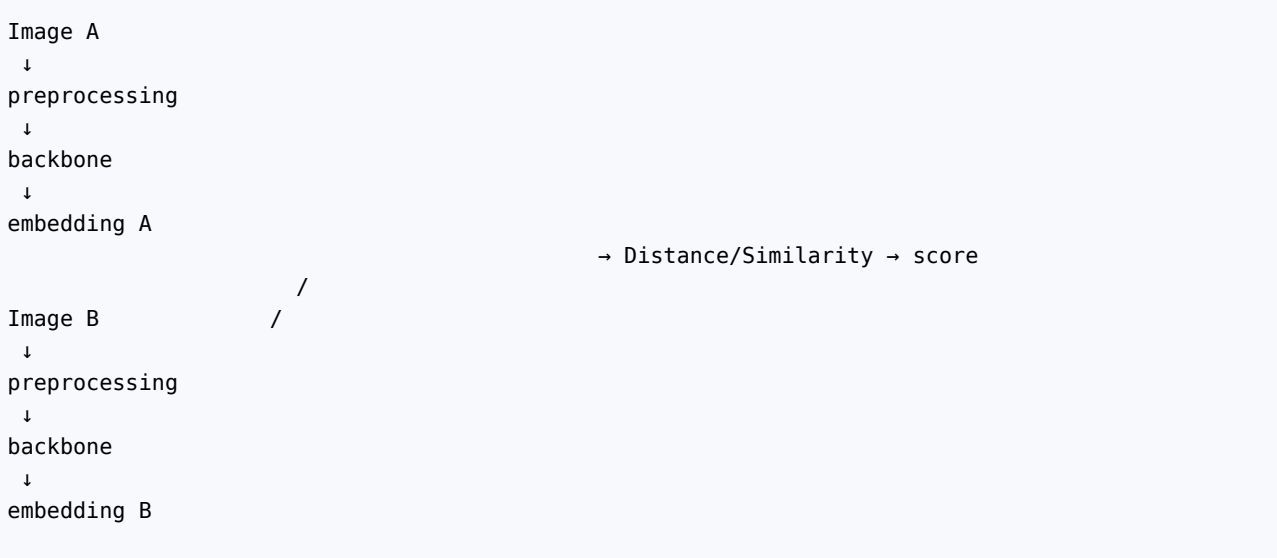
Evaluation transforms validation scores into metrics that help characterize the model. FAR, FRR, EER, threshold, and metric orientation were already defined in Sections 7, 19, and 38–40.

In SFAAO, evaluation may produce:

- EER and corresponding threshold;
- number of matches and non-matches;
- FAR×FRR curve;
- score distribution;
- history by epoch;
- identification of the best state;
- paths and hashes of generated artifacts.

These results describe the model under the evaluated population. They do not eliminate subsequent calibration linked to the case or scientific protocol.

4.9 Detailed inference pipeline



The score must not be interpreted in isolation; its decision depends on the corresponding calibration.

Chapter 5

Forensic protocols, calibration and decisions

5.1 Forensic case

The case is the central unit of the platform.

A case contains:

- identifier;
- request data;
- examiner;
- RC;
- RO;
- questioned signatures;
- calibrations;
- executions;
- results;
- reports.

5.2 Signature types: RC, RO, and QUESTIONED

SFAAO separates three sets because each has a different forensic origin and function. This distinction must be understood before the calibration and comparison pipelines.

RC — references collected in the contractual pattern

What they are. Signatures collected directly from the client/signatory during the forensic procedure. **How they are produced.** The signatory signs while reproducing the pattern habitually used in contracts. **What they are used for.** To provide known standards representing the natural variability of the contractual pattern, form genuine pairs, calibrate, and compare against QUESTIONED signatures. **Critical rule.** RC is not a signature extracted from the contract under examination.

RO — official references

What they are. Signatures also collected directly from the client/signatory. **How they are produced.** The signatory signs while reproducing the pattern used in official documents, such as an RG, CNH, or other identification documents in which that pattern is used. **What they are used for.** To provide a second known reference set, with an official graphic pattern, for its own calibration and comparison against QUESTIONED signatures. **Critical rule.** RO does not simply mean copying the image from an RG or CNH; the RO used in the procedure is collected from the signatory reproducing that pattern.

QUESTIONED — questioned signatures

What they are. All signatures present in the contract submitted for examination. **What they are used for.** They constitute the material whose authorship/compatibility is under evaluation. **Critical rule.** No signature from the contract is initially RC or RO; all remain QUESTIONED throughout case traceability, even after receiving a forensic classification.

```
SIGNATORY
   ├─ collection reproducing contractual pattern → RC
   └─ collection reproducing official pattern    → RO

CONTRACT UNDER EXAMINATION
   └─ all signatures → QUESTIONED
```

5.2.1 Human forensic classification of QUESTIONED signatures

In the forensic protocol, the forensic handwriting examiner examines the QUESTIONED signatures and assigns the forensic role required by the procedure. This classification **is not inferred automatically by the model**.

QUESTIONED_COMPATIBLE

A QUESTIONED signature classified by the forensic handwriting examiner as compatible with the signatory's pattern. It remains part of the QUESTIONED set and does not become RC or RO.

QUESTIONED_IMPOSTOR

A QUESTIONED signature classified by the examiner as divergent from or incompatible with the signatory's pattern and used, when defined by the FORENSIC_CASE protocol, as a non-match for calibration. It remains a questioned signature from the contract.

Didactic example: a contract contains 14 signatures. All enter the case as QUESTIONED. After the handwriting examination, the examiner may classify 7 as compatible and 7 as impostors. This human classification precedes forensic calibration.

```
14 QUESTIONED signatures from the contract
          ↓
analysis by the forensic handwriting examiner
          ↓
 ├─ 7 QUESTIONED_COMPATIBLE
 └─ 7 QUESTIONED_IMPOSTOR
          ↓
forensic calibration
          ↓
RC×QUESTIONED and RO×QUESTIONED comparisons
```

Difference between the forensic and scientific protocols

In `FORENSIC_CASE`, human classification may participate in the formation of calibration populations according to the documented pipeline. In `SCIENTIFIC_EXTERNAL`, QUESTIONED signatures remain outside threshold estimation; non-matches come from external impostors, and QUESTIONED signatures are subsequently used in inference/comparison.

5.3 Statistical calibration

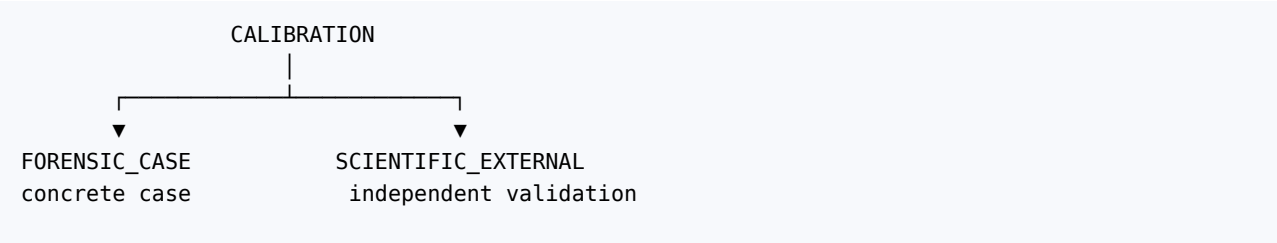
Calibration builds two populations:

```
MATCH
NON-MATCH
```

and calculates:

- FAR;
- FRR;
- EER;
- τ_EER;
- calibrated threshold;
- operational τ.

5.4 Dual adaptive calibration



5.5 Detailed FORENSIC_CASE pipeline — RC

Input:

```
n RC references
questioned signatures classified by the examiner
```

Genuine:

```
C(n,2)
```

With five RC references:

```
C(5,2)=10
```

Non-match:

```
QUESTIONED_IMPOSTOR × RC
```

With seven questioned impostor signatures:

```
7×5=35
```

Critical rule: `QUESTIONED_COMPATIBLE` is not promoted to the genuine population. Questioned material remains questioned material.

5.6 Detailed FORENSIC_CASE pipeline — RO

Genuine:

```
C(n_RO,2)
```

With five RO references:

```
10
```

Non-match:

```
all questioned signatures × RO
```

With 14 questioned signatures and five RO references:

```
14×5=70
```

This behavior is part of the established RO forensic protocol.

5.7 Detailed SCIENTIFIC_EXTERNAL pipeline [16]

The **scientific case contains the same three documentary sets as the forensic case**:

```
RC
RO
QUESTIONED
```

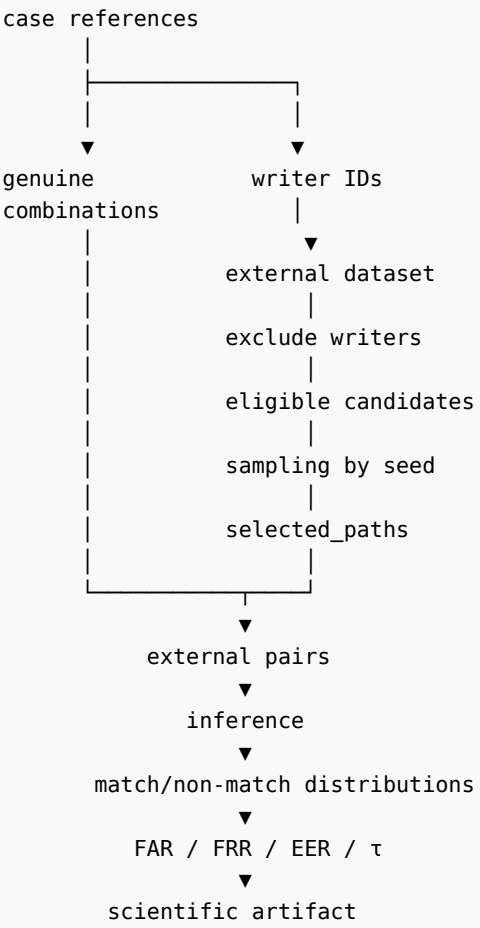
The difference is not the existence of these sets in the case, but **the methodological role of each one during calibration**.

- **RC**: may constitute the reference set for the RC scientific scenario;
- **RO**: may constitute the reference set for the RO scientific scenario;
- **QUESTIONED**: remains registered in the case and is used later in inference/evaluation, **but does not participate in scientific threshold estimation**.

Thus, the complete scientific-protocol flow is:

```
CASE
 ├─ RC
 ├─ RO
 └─ QUESTIONED
       │
       ├──────────────────────────────────────────┐
       │                                          │
       ▼                                          ▼
SCIENTIFIC CALIBRATION                    FORENSIC INFERENCE
uses RC or RO + external impostors        uses RC/RO + QUESTIONED
QUESTIONED excluded from threshold        applies calibrated threshold
```

This distinction is essential: **the questioned signature exists in the scientific case, just as in the forensic case, but it is kept outside the population used to estimate the scientific threshold**, preserving independence between calibration and evaluation [16].



Questioned signatures **do not enter** scientific threshold estimation. This separation between calibration and forensic evaluation is a central methodological principle of the scientific protocol associated with SFAAO [16].

5.7.1 RC scientific scenario

```
case RC
 ↓
RC×RC combinations → genuine population
 ↓
RC×external impostors → non-match population
 ↓
FAR / FRR / EER / threshold
 ↓
QUESTIONED remains reserved for inference
```

With five RC references:

```
C(5,2) = 10 genuine pairs
```

5.7.2 RO scientific scenario

```
case RO
 ↓
RO×RO combinations → genuine population
 ↓
RO×external impostors → non-match population
 ↓
FAR / FRR / EER / threshold
 ↓
QUESTIONED remains reserved for inference
```

With five RO references:

```
C(5,2) = 10 genuine pairs
```

5.7.3 Role of QUESTIONED signatures in the scientific protocol

The `QUESTIONED` signatures:

```
exist in the case
are displayed and managed normally
are used in the comparison/inference stage
are not used to estimate scientific τ_EER
do not enter external-impostor selection
do not alter the calibration population
```

Therefore:

```
FORENSIC_CASE       → RC + RO + QUESTIONED
SCIENTIFIC_EXTERNAL → RC + RO + QUESTIONED
```

In both cases, the three documentary sets belong to the case. The difference lies **in how QUESTIONED signatures participate — or do not participate — in forming calibration populations**.

With five references and 2,000 impostors:

```
genuine = 10
non-match = 2,000×5 = 10,000
```

With five references and 100 impostors:

```
genuine = 10
non-match = 100×5 = 500
```

5.8 Scientific provenance [16]

The external selection may record:

```
dataset_name
dataset_profile
dataset_root
dataset_version
profile_name
profile_version
selection_method
seed
excluded_writers
eligible_count
requested_count
selected_count
selection_hash_sha256
selected_paths
```

The SHA-256 hash makes it possible to identify the exact selection used. This record complements the scientific requirement that the external population be identifiable and reproducible [16].

5.9 FAR, FRR, and EER

All three metrics depend on the **MATCH** (genuine) and **NON-MATCH** (impostor) populations, the threshold τ, and the direction of the comparison metric [5].

5.9.1 FAR — False Acceptance Rate [5]

Formal definition. FAR is the proportion of non-matches that are incorrectly accepted:

$$FAR(\tau) = \frac{\#\{\text{non-matches accepted at } \tau\}}{N_{nonmatch}}.$$

In SFAAO:

$$FAR_{cos}(\tau) = \frac{\#\{s_i^I \geq \tau\}}{N_I}, \qquad FAR_E(\tau) = \frac{\#\{d_i^I \leq \tau\}}{N_I}.$$

Thus, the false-acceptance error uses $\geq$ for cosine similarity and $\leq$ for Euclidean distance.

5.9.2 FRR — False Rejection Rate [5]

Formal definition. FRR is the proportion of matches that are incorrectly rejected:

$$FRR(\tau) = \frac{\#\{\text{matches rejected at } \tau\}}{N_{match}}.$$

In SFAAO:

$$FRR_{cos}(\tau) = \frac{\#\{s_i^G < \tau\}}{N_G}, \qquad FRR_E(\tau) = \frac{\#\{d_i^G > \tau\}}{N_G}.$$

5.9.3 EER — Equal Error Rate [5]

Formal definition. EER is the rate associated with the point at which FAR and FRR are equal or, with discrete data, the threshold that minimizes their difference:

$$\tau_{EER} = \arg\min_{\tau} \left| FAR(\tau) - FRR(\tau) \right|.$$

EER must be interpreted considering the dataset, protocol, model, metric, and sampling used. An EER equal to zero in one calibration means perfect separation **in that evaluated population**; it does not imply a universal error rate of zero.

5.10 Thresholds, FAR target, and dataset shift [15], [16]

SFAAO distinguishes four concepts that must not be confused:

- τ_{model} — threshold obtained from model evaluation in the training/validation context; it remains model-provenance information and does not, by itself, determine the case decision;
- $\tau_{EER,cal}$ — threshold estimated at the point where FAR and FRR are closest in the calibration population;
- τ_{FAR} — threshold selected by calibration under the empirical FAR-target constraint;
- $\tau_{operational}$ — threshold actually applied to the technical decision.

In SFAAO 2.4.0, the official calibration interface fixes `CalibrationWindow.TARGET_FAR = 0.01`. Therefore, the operational FAR target used by the GUI is

$$\alpha = 0.01 = 1\%.$$

This parameter belongs to **calibration**, not training. It is passed to `CalibrationConfig` as `target_far`; the official interface does not expose a control that allows the user to modify this value.

For cosine similarity, the constraint is evaluated as

$$\widehat{FAR}_{cos}(\tau) = \frac{\#\{s_i^I \geq \tau\}}{N_I} \leq 0.01.$$

For Euclidean distance,

$$\widehat{FAR}_{E}(\tau) = \frac{\#\{d_i^I \leq \tau\}}{N_I} \leq 0.01.$$

Calibration searches among the evaluated thresholds for a τ_{FAR} satisfying this empirical constraint. This process **does not force EER to be less than or equal to 1%**. EER is computed independently as

$$\tau_{EER,cal} = \arg\min_{\tau} \left| \widehat{FAR}(\tau) - \widehat{FRR}(\tau) \right|.$$

The original training threshold does not replace case-specific calibration. The motivation is **dataset shift**: score distributions during training may differ from those encountered in the operational case [15], [16].

5.10.1 Cosine similarity: higher values are more genuine

Because acceptance occurs at high values, a more conservative operational policy moves the threshold upward. When τ_{FAR} and $\tau_{EER,cal}$ are available:

$$\boxed{\tau^{cos}_{operational} = \max\left(\tau_{EER,cal}, \tau_{FAR}\right)}.$$

The operational policy uses three zones. For cosine similarity:

$$\begin{cases} \text{COMPATIBLE}, & s \geq \tau^{cos}_{operational}, \\ \text{INCONCLUSIVE}, & \tau_{EER,cal} \leq s < \tau^{cos}_{operational}, \\ \text{INCOMPATIBLE}, & s < \tau_{EER,cal}. \end{cases}$$

5.10.2 Euclidean distance: lower values are more genuine

For Euclidean distance, the orientation is reversed: lower values indicate greater proximity. Therefore, a more conservative operational policy moves the threshold downward. When both thresholds are available:

$$\boxed{\tau^{E}_{operational} = \min\left(\tau_{EER,cal}, \tau_{FAR}\right)}.$$

The operational policy uses three zones. For Euclidean distance:

$$\begin{cases} \text{COMPATIBLE}, & d_E \leq \tau^{E}_{operational}, \\ \text{INCONCLUSIVE}, & \tau^{E}_{operational} < d_E \leq \tau_{EER,cal}, \\ \text{INCOMPATIBLE}, & d_E > \tau_{EER,cal}. \end{cases}$$

Statistical interpretation of the FAR target

The requirement $\widehat{FAR} \leq 1\%$ is an **empirical constraint** observed in the calibration population. It is not a mathematical guarantee that the true FAR in every future population will remain less than or equal to 1%. Its interpretation depends on the impostor population, sample size, dependence among comparisons, model, metric, and operating conditions.

Semantic-safety rule

The rule "higher score is better" must never be reused automatically. SFAAO records the metric and its orientation: cosine uses `higher_is_genuine=True`; Euclidean uses `higher_is_genuine=False`. Decisions, charts, FAR/FRR, and result persistence must respect this semantics.

5.11 Three-zone operational policy and threshold interval

In SFAAO 2.4.0, the interval between τ_{EER} and $\tau_{operational}$ is an explicit part of the computational decision. The policy is no longer binary and distinguishes **COMPATIBLE**, **INCONCLUSIVE**, and **INCOMPATIBLE**.

For cosine similarity, when $\tau^{cos}_{operational} \geq \tau_{EER,cal}$:

$$\begin{cases} \text{COMPATIBLE}, & s \geq \tau^{cos}_{operational}, \\ \text{INCONCLUSIVE}, & \tau_{EER,cal} \leq s < \tau^{cos}_{operational}, \\ \text{INCOMPATIBLE}, & s < \tau_{EER,cal}. \end{cases}$$

For Euclidean distance, when $\tau^{E}_{operational} \leq \tau_{EER,cal}$:

$$\begin{cases} \text{COMPATIBLE}, & d_E \leq \tau^{E}_{operational}, \\ \text{INCONCLUSIVE}, & \tau^{E}_{operational} < d_E \leq \tau_{EER,cal}, \\ \text{INCOMPATIBLE}, & d_E > \tau_{EER,cal}. \end{cases}$$

When $\tau_{operational} = \tau_{EER}$ there is no intermediate zone. Under LORO, this rule is applied independently to each A(N-1) fold; folds are not averaged again and are not subjected to majority voting.

5.11.1 Aligned statistical domain: A1, A(N-1), and A(N)

The current operational SFAAO explicitly distinguishes three score levels. **A1** is the pairwise score between one reference and one questioned signature. Under LORO calibration and inference, A1 remains an auditable component and provenance element, but it is not the final statistical decision unit.

With N references, the decision domain is **A(N-1)**. The `LEAVE_ONE_REFERENCE_OUT` strategy excludes one reference per fold and computes the mean score against the remaining $N - 1$ references. Thus, with five references, calibration and inference use **A4** and produce five folds.

The mean against all N references, **A(N)**, remains available as a descriptive statistic. It does not receive the LORO threshold and does not replace the A(N-1) folds.

The corresponding operational configuration is:

```
decision_statistic = LORO_MEAN
fold_strategy      = LEAVE_ONE_REFERENCE_OUT
aggregation_size   = reference_count - 1
fold_count         = reference_count
```

Therefore, when $N = 5$:

```
A1 = pairwise score / provenance
A4 = LORO decision unit
A5 = descriptive statistic
folds = 5
```

5.11.2 Thresholds and three-zone operational policy

SFAAO distinguishes three threshold concepts:

- τ_{EER}: threshold corresponding to the EER operating point of the current calibration;
- $\tau_{forensic}$: threshold associated with the configured target FAR;
- $\tau_{operational}$: final threshold actually used by the operational policy.

For similarity metrics such as Cosine Similarity:

$$\tau_{operational} = \max(\tau_{forensic}, \tau_{EER}).$$

The rule is:

```
score >= tau_operational
    -> COMPATIBLE

tau_EER <= score < tau_operational
    -> INCONCLUSIVE

score < tau_EER
    -> INCOMPATIBLE
```

For distance metrics such as Euclidean Distance:

$$\tau_{operational} = \min(\tau_{forensic}, \tau_{EER}).$$

The rule is:

```
score <= tau_operational
    -> COMPATIBLE

tau_operational < score <= tau_EER
    -> INCONCLUSIVE

score > tau_EER
    -> INCOMPATIBLE
```

When $\tau_{operational} = \tau_{EER}$, there is no intermediate inconclusive zone.

Under LORO, the rule above is applied **individually to each A(N-1) fold**.

5.11.3 Conservative fold consolidation

LORO folds are not averaged again to produce a second decision, and majority voting is not used. Consolidation is based on unanimity:

- all `COMPATIBLE`: `STABLE_COMPATIBLE`;
- all `INCOMPATIBLE`: `STABLE_INCOMPATIBLE`;
- all `INCONCLUSIVE`: `STABLE_INCONCLUSIVE`;
- any mixture: `REFERENCE_SENSITIVE`, with consolidated `INCONCLUSIVE`.

This design makes reference-set sensitivity explicit instead of hiding it by averaging or majority voting.

5.11.4 Calibration–inference alignment

Schema v6 records the calibration domain and inference domain separately. The `alignment_status` field may be:

```
ALIGNED
MISMATCH
LEGACY_MISMATCH
```

New operational LORO executions must be `ALIGNED`. `MISMATCH` indicates statistical misalignment. Historical records in which an A1 threshold was applied to an aggregated statistic remain marked as `LEGACY_MISMATCH` and are not silently reinterpreted.

5.11.5 RC, RO, and scientific protocol semantics

Under `SCIENTIFIC_EXTERNAL`, case QUESTIONED signatures do not participate in threshold estimation; non-matches come from external impostors, and QUESTIONED evaluation occurs later.

Under `FORENSIC_CASE` with RC, only QUESTIONED signatures previously classified by the examiner as `QUESTIONED_IMPOSTOR` may compose the non-match population. Compatible QUESTIONED signatures are not promoted to genuine, and unclassified QUESTIONED signatures do not silently enter that population.

In the RO context, QUESTIONED signatures are intentionally treated as non-match against official references, according to the operational definition of that protocol.

5.11.6 Fold persistence and auditable decision

In schema v6, `questioned_results` records the descriptive statistic, decision domain, fold counts, stability state, and consolidated decision. Each fold is persisted in `questioned_decision_folds`, including the excluded reference, A(N-1), τ_{EER}, $\tau_{operational}$, and technical decision.

Persistence of the QUESTIONED result and its folds is atomic: either the result and every fold are stored, or the transaction is rolled back. An operational LORO result must not exist with only a partial fold set.

5.12 Detailed comparison pipeline

```
01. open case
02. choose active model
03. resolve selected calibration
04. choose RC or RO
05. load references
06. load questioned signatures
07. for each questioned signature:
08.     for each reference:
09.         preprocess
10.         infer embeddings
11.         calculate score
12.         persist score
13. preserve pairwise A1 as provenance
14. construct LORO A(N-1) folds
15. validate alignment_status = ALIGNED
16. apply tau_EER / tau_operational per fold
17. persist questioned_decision_folds
18. consolidate STABLE_* or REFERENCE_SENSITIVE
19. persist ComparisonRun + questioned_results
20. keep A(N) as descriptive statistic only
```

With five references and 14 questioned signatures:

```
70 individual scores
14 aggregates
```

5.13 Technical decision versus forensic classification [16]

```
FORENSIC EXAMINER CLASSIFICATION
           ≠
COMPUTATIONAL DECISION
```

A signature classified by the examiner as compatible may receive an incompatible computational decision. The system does not copy the human classification. In the scientific protocol, questioned signatures remain outside calibration and are reserved for evaluation [16].

Chapter 6

Traceability, auditing and security

6.1 Traceability

Each result must make it possible to reconstruct:

```
user
 → case
 → signature
 → model
 → configuration
 → calibration
 → threshold
 → ComparisonRun
 → scores
 → aggregate
 → report
```

6.2 Data hierarchy

```
User/Examiner
   ↓
Case
   ↓
Signatures
   ↓
Model
   ↓
Calibration
   ↓
Comparison
   ↓
Results
   ↓
Report
```

Calibration belongs simultaneously to the **case** and the **model**. It is never global.

6.3 Model as a scientific artifact

A model is not merely a weights file.

Its package must include, when applicable:

```
model weights
model_config.json
training_config.json
```

```
history
evaluation metrics
EER
threshold
plots
metadata
```

Each model must be treatable as an independent scientific experiment.

6.4 Auditing and integrity

Relevant classification changes must record:

```
old value
new value
actor
timestamp
```

Example:

```
QUESTIONED_COMPATIBLE
→ QUESTIONED_IMPOSTOR
→ QUESTIONED_COMPATIBLE
```

Integrity prevents silent removal of data that have already been used.

6.5 Deletion rules

- selected calibration or calibration used in comparison: block;
- signature used by results/comparisons: block;
- case with historical evidence: preserve;
- physical deletion only when safe;
- storage artifacts must be removed only after successful completion of the database operation.

6.5.1 Data-access policy

The architectural specification establishes independence between users and states that one user must not automatically obtain information belonging to another. Concrete implementation of this policy must respect the institutional configuration and authorization rules defined for the deployment. This rule is distinct from the separation of **roles**:

```
ADMIN    → governs the platform
EXAMINER → performs authorized forensic operations
```

The hiding of administrative functions from EXAMINER users was validated for release 2.4.0. Additional case-segregation policies by user or organizational unit must be validated according to the institutional deployment.

6.6 Security

6.6.1 Authentication

Passwords are not stored in plaintext. SFAAO uses **PBKDF2-HMAC-SHA256**, a key-derivation mechanism that applies many cryptographic iterations to a password before persisting its verifier. A **random salt** is a distinct value associated with each password to hinder precomputed tables and attacks based on repeated hashes. `compare_digest` performs a comparison resistant to simple execution-time differences. These mechanisms reduce the risk of direct credential exposure if the database is accessed improperly.

Login-attempt control and temporary lockout are also implemented to hinder repeated password-guessing attacks.

6.6.2 ADMIN

Governs:

- users;
- datasets;
- models;
- training;
- activation;
- administrative operations.

6.6.3 EXAMINER

Operates:

- cases;
- signatures;
- calibration;
- comparison;
- reports.

Administrative menus must not be visible.

Principle:

```
THE EXAMINER USES.
THE ADMINISTRATOR GOVERNS.
```

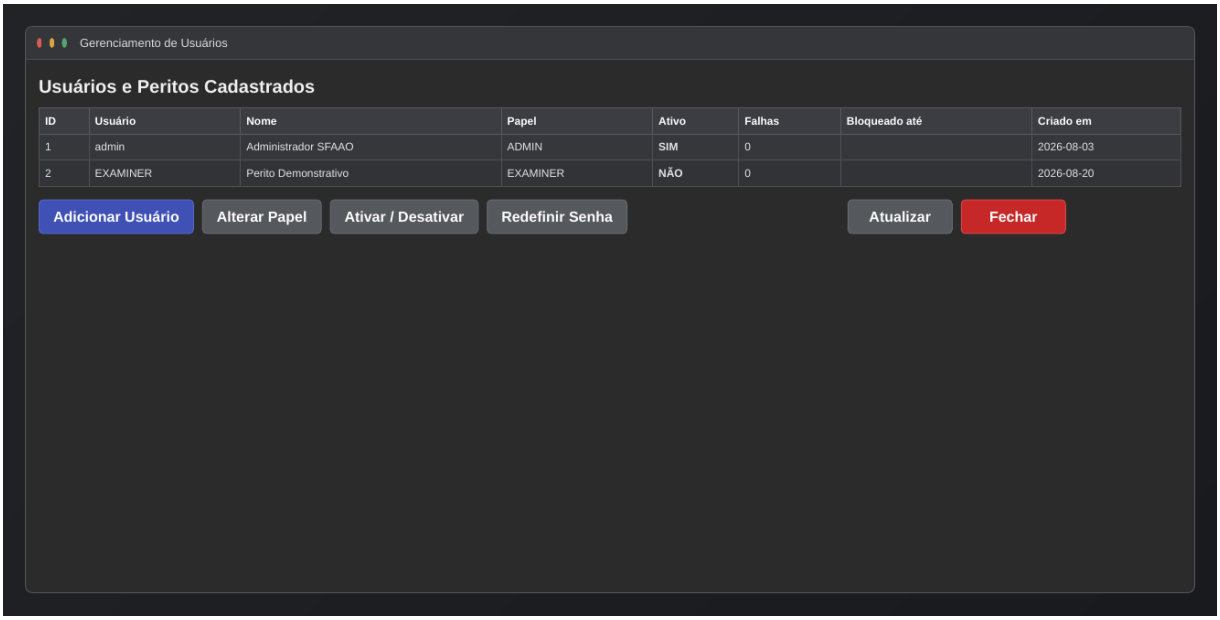


Figure 6.1: User and examiner management showing accounts, roles, activation status, and administrative actions.

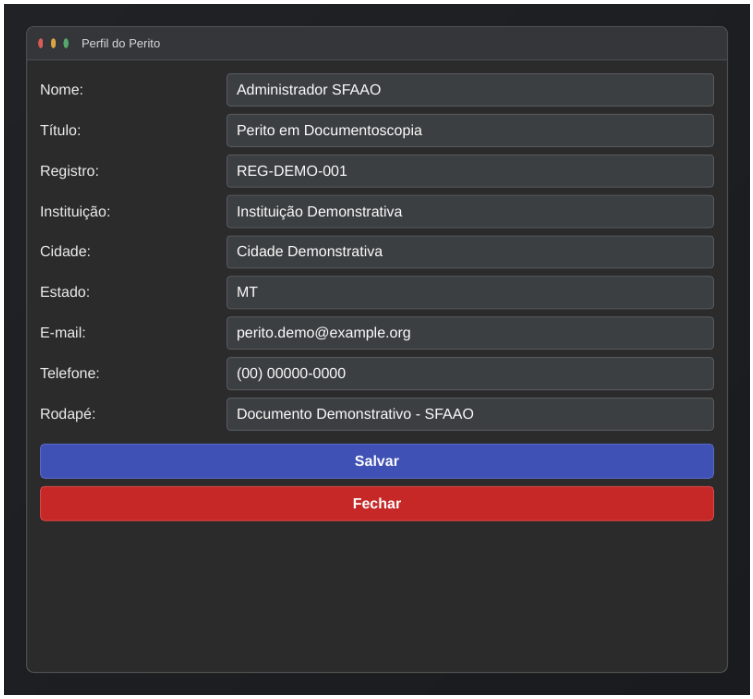


Figure 6.2: Examiner professional-profile form, including identification and contact details.

6.6.4 Security and integrity bootstrap

Before any authentication screen is displayed, `SecurityBootstrap` executes a chain of security checks. The consolidated sequence in the final release includes storage protection, cryptographic license validation, binding to machine identity, validation of the protected time state, clock-rollback protection, and verification of the integrity manifest using SHA-256. A critical failure stops startup before the GUI is opened.

The security bootstrap runs entirely in the background, with no dedicated screen or window. Users interact with the application only after the internal checks have completed.

6.7 First administrator

Clean installation:

```
0 users
 ↓
FirstAdminWindow
 ↓
create an ADMIN
 ↓
conventional login
```

There are no embedded default credentials.

6.8 Storage security

Storage is the persistent area that holds the database, cases, images, models, calibrations, results, reports, license, and security state. Because these items may have forensic value, protecting them is part of the SFAAO architecture.

On POSIX systems, the policy uses restrictive permissions, such as `umask 077`, directories with mode 700, and files with mode 600 where applicable.

In the Windows 2.4.0 architecture, persistence must be maintained at:

```
C:
ProgramData
SFAAO
```

Installation under `C:`
`Program Files`
`SFAAO` uses junctions to redirect mutable directories to `ProgramData`:

```
storage          → C:
ProgramData
SFAAO
storage
models/default   → C:
ProgramData
SFAAO
models
default
models/custom    → C:
ProgramData
SFAAO
models
custom
security/state   → C:
ProgramData
SFAAO
security
state
```

A **junction** is a Windows filesystem link that makes one path point to another directory. It allows the executable to remain under Program Files while persistent data is kept under ProgramData without changing the paths expected by the application.

The Windows 2.4.0 installer applied ACL protection to persistent application data. The policy avoids broad access by `Users` or `Everyone`, granting only required permissions to authorized accounts. Actual ACLs must be rechecked on each destination computer.

The SQLite database, WAL/SHM files, and other persistent artifacts are subject to the same protection policy. Data preservation across upgrade/reinstallation was observed in the validated Windows 11 environment; repeat this check at each institution.

Chapter 7

Reports and operational workflows

7.1 Detailed report-generation pipeline

In the final release, report generation combines technical-scientific assembly, versioning, artifact integrity, and transactional **quota** governance. A quota is the quantitative usage limit defined by the license for a given resource. In this flow, reservation and consumption are recorded transactionally to avoid inconsistencies. The consolidated flow is:

```
01. selected case and protocol
02. validate feature/format permitted by the license
03. build the current forensic context
04. calculate fingerprint and SHA-256 logical key for the context
05. search for a reusable logical report
06. BEGIN IMMEDIATE
07. if new report: validate quota + reserve version + record ledger entry
08. if complementary format: reuse report_uid/version without new quota
09. ReportPipeline loads case, examiner, model, calibrations, and comparisons
10. generate DOCX or PDF
11. rebuild context and verify that no concurrent change occurred
12. version the physical artifact
13. calculate the file SHA-256
14. bind path/hash to the report record
15. on failure without a valid artifact: mark FAILED + release quota idempotently
```

Reservation of a new report and the quota ledger entry occur in the same transaction. Generation of a complementary format for the same logical report — for example, PDF after DOCX, or DOCX after PDF — reuses the same `report_uid` and the same version, without new consumption. When a new report fails before a valid artifact is associated with it, the record remains in the `FAILED` state for traceability and the corresponding quota reservation is released idempotently. Generation of a complementary format for the same logical report reuses the same `report_uid` and does not create new consumption.

The report may contain identification, request, methodology, model, dataset, backbone, loss, metric, embedding, dropout, original metrics, RC/RO calibrations, FAR/FRR/EER, thresholds, UIDs, consolidated results, executive summary, automated findings, computational synthesis, and appendices. The computational synthesis does not replace the examiner's conclusion.

Statistical report provenance. The current report pipeline consumes persisted results and validates statistical alignment before document materialization. Statistical provenance metadata uses `schema_version 3`. The logical fingerprint includes, among other elements, calibration and inference domains, aggregation size, fold strategy, reference count, fold count, τ_{EER}, $\tau_{operational}$, A(N) results, A(N-1) folds, fold counts, `stability_status`, and the consolidated decision. Changes to these decision-relevant elements change the logical report context.

7.2 Complete operational flow

```
INSTALLATION
 ↓
FIRST ADMIN
 ↓
LOGIN
 ↓
USER REGISTRATION
 ↓
DATASET REGISTRATION/VALIDATION
 ↓
MODEL TRAINING OR IMPORT
 ↓
MODEL ACTIVATION
 ↓
CASE CREATION
 ↓
EXAMINER REGISTRATION
 ↓
RC UPLOAD
 ↓
RO UPLOAD
 ↓
QUESTIONED UPLOAD
 ↓
FORENSIC CLASSIFICATION
 ↓
FORENSIC OR SCIENTIFIC CALIBRATION
 ↓
RC COMPARISON
 ↓
RO COMPARISON
 ↓
RESULT REVIEW
 ↓
DOCX/PDF GENERATION
 ↓
ARCHIVING
```

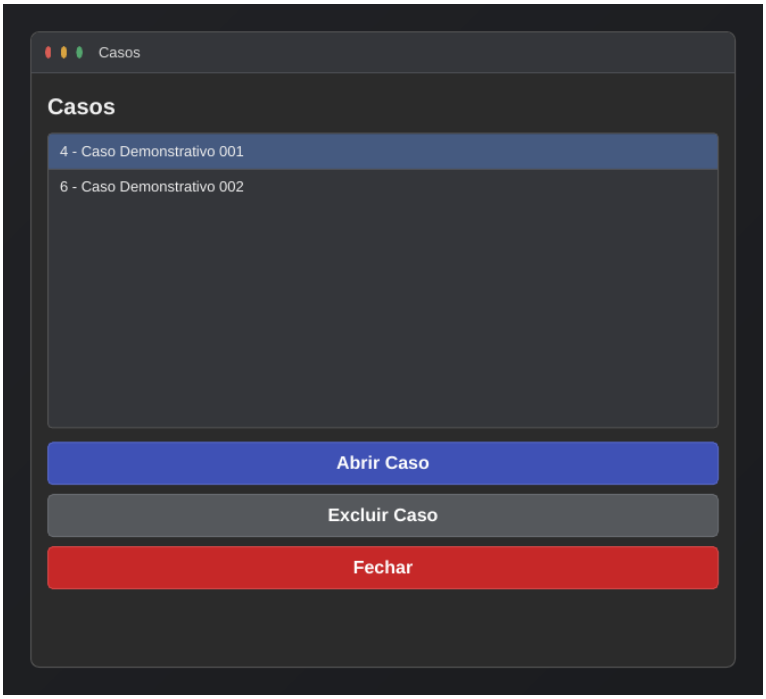


Figure 7.1: Case list with actions to open or delete a case and close the window.

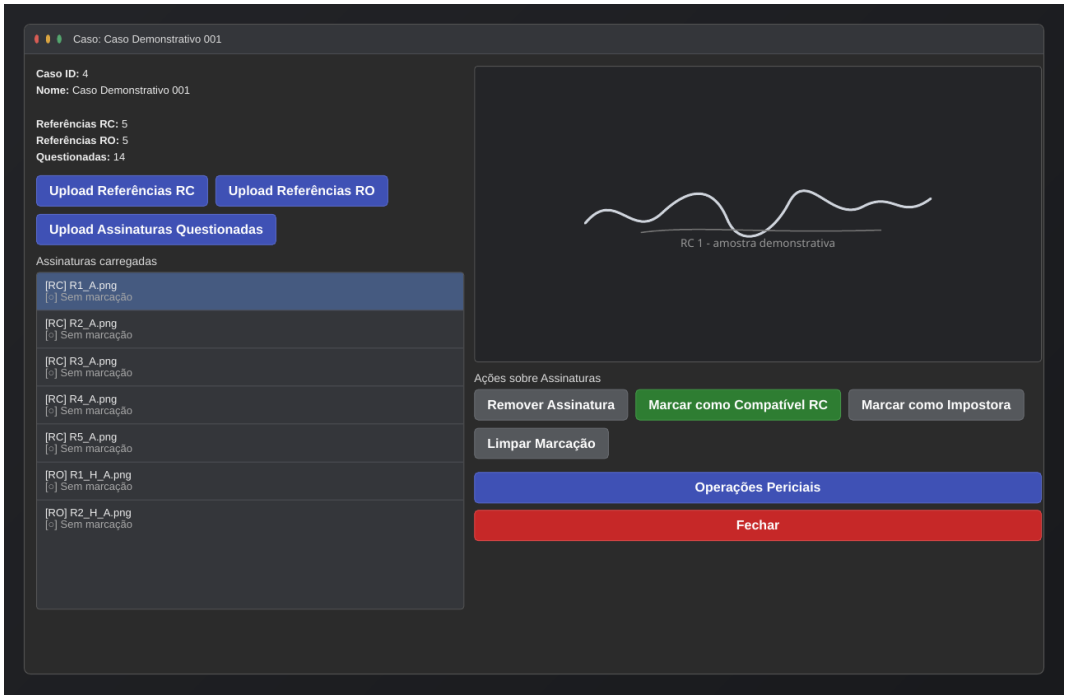


Figure 7.2: Case signatures window with reference and questioned samples, preview, and file-management actions.



Figure 7.3: Case operations menu: report details and generation, calibration, comparison, and batch/CSV results.

7.2.1 Application startup pipeline

```
app.py
 ↓
SecurityBootstrap
 ↓
storage protection / secure permissions
 ↓
license validation and LicenseContext creation
 ↓
MachineFingerprint V2
 ↓
SecureTimeState + TimeGuard
 ↓
IntegrityChecker + SHA-256 manifest
 ↓
SQLite schema initialization/migration
 ↓
user count
 ├─ 0 users → FirstAdminWindow
 └─ ≥1 user → LoginWindow
 ↓
authentication + ADMIN/EXAMINER resolution
 ↓
Main Window with corresponding permissions
```

The GUI is started only after the critical security validations are approved. Fingerprint V2 avoids dependence on the hostname or MAC address and uses stable operating-system identifiers combined with platform and architecture before SHA-256 derivation.

7.2.2 Model-governance pipeline

```
ADMIN
 ↓
Model Manager
 ↓
import package OR train new model
 ↓
validate structure/configuration
 ↓
register in catalog
 ↓
set as active
 ↓
model available to operational flows
```

Activation does not change historical results. Previous comparisons remain linked to the model and calibration originally used.

7.2.3 Dataset-governance pipeline

```
ADMIN
 ↓
Dataset Manager
 ↓
add dataset
 ↓
select profile
 ↓
validate classification and structure
 ↓
persist catalog
 ↓
dataset enabled for training/scientific calibration
```

Removing a dataset from the catalog does not imply redistribution or physical deletion of a third-party dataset.

7.2.4 Licensing, profiles, and quotas

A **license** defines which resources and limits the installation may use. A **quota** is the quantitative limit for a given resource. The final release has three operational profiles: `TRIAL`, `ACADEMIC`, and `FULL`. `LicenseContext` is the internal structure that gathers the valid permissions after the license has been verified. The **fail-closed** policy means that if a required limit is absent or invalid, the system blocks the operation instead of assuming permission. The internal value `None`, when provided by a valid profile, represents an unlimited resource.

Resource	TRIAL	ACADEMIC	FULL
Active users	1	10	unlimited
Active EXAMINER examiners	1	5	unlimited
Cases	2	50	unlimited
Individual comparisons	unlimited	unlimited	unlimited

Resource	TRIAL	ACADEMIC	FULL
Batch executions	3	100	unlimited
Reports	1	20	unlimited
CSV exports	3	50	unlimited
Training runs	3	100	unlimited
Calibrations	2	50	unlimited
Available datasets	2	10	unlimited
Registered models	3	20	unlimited

Regarding protocols, TRIAL allows scientific and forensic protocols; ACADEMIC allows the scientific protocol and blocks the forensic protocol by policy; FULL allows both. The `audit_trail` feature is enabled only in FULL. Training, calibration, comparison, batch analysis, and DOCX/PDF generation remain enabled in all three profiles, subject to the applicable quotas.

7.2.4.1 Quota semantics

Quotas are classified into three groups. A **usage ledger** is a transactional record of consumption already performed, similar to an accounting ledger. Cumulative resources — `training_runs`, `calibrations`, `batch_executions`, `reports`, and `csv_exports` — are accounted for by this ledger. Current-state resources — `users`, `examiners`, `datasets`, and `models` — are measured directly in the database. Cases use a hybrid rule, taking the higher value between the ledger and current state, preventing simple deletion from retroactively reducing authorized consumption.

The SFAAO 2.4.0 operational policy uses **transactional reservation with idempotent rollback** for cumulative resources. **Rollback** means undoing reserved consumption when the operation fails without producing the corresponding valid result. **Idempotent** means that repeating the release of the same reservation does not cause negative consumption or a duplicate effect. The quota is authorized before the operation, but an operational failure that does not produce the corresponding valid artifact/result releases the applicable reservation without deleting the failure record required for traceability.

In the validated release, usage is released idempotently for training, calibration, batch execution, and CSV export when the operation fails. For reports, a new report that fails before obtaining a valid artifact remains `FAILED` and releases the quota; creation of the complementary DOCX/PDF format for the same logical report does not consume quota again. Cancellations that occur before authorization, such as choosing the CSV destination, also do not generate consumption.

This design avoids charging quota for an operation that did not produce the expected result, without losing the audit trail of the attempt.

During final validation on Windows 11, two training runs that were started and subsequently canceled were tested and **did not consume** the `training_runs` quota. This evidence confirms cancellation rollback in the two tests performed. This Manual does not generalize that observation to deep failure paths that were not explicitly exercised.

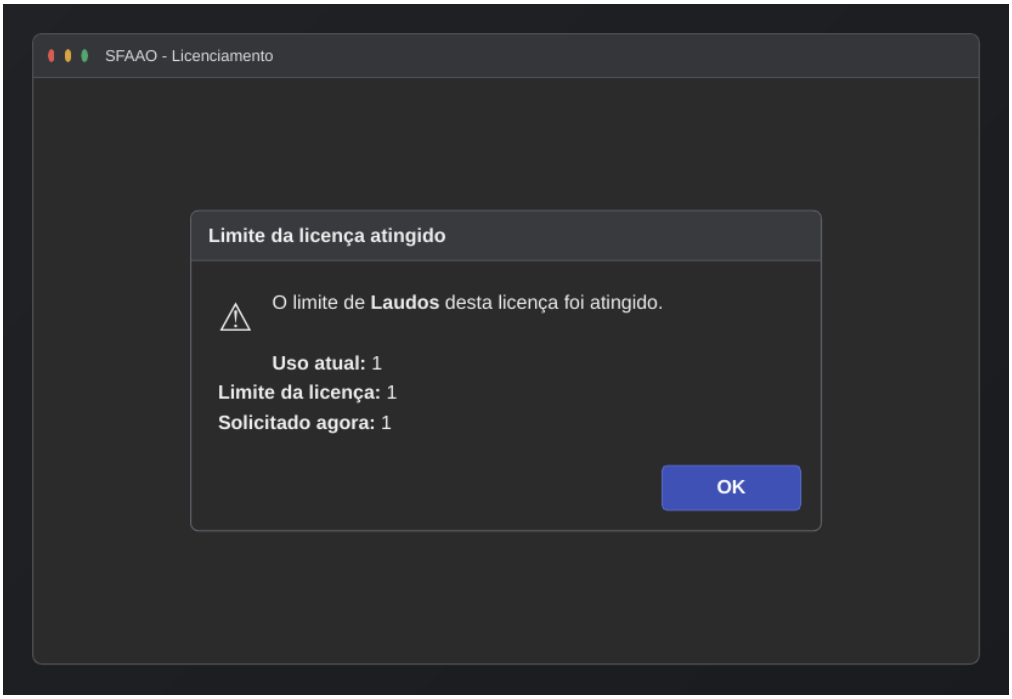


Figure 7.4: License-quota warning showing the affected resource, current usage, and permitted limit.

7.2.5 Startup and continuity security

The license, machine identity, protected time state, and code integrity are verified before operation. The protected manifest records SHA-256 hashes of monitored files; modified or missing files stop startup.

For release 2.4.0, the protected Linux payload includes internal integrity checks. The four installed packaging artifacts were checked with `sha256sum -c SHA256SUMS` under `/opt/sfaao`, all with `OK` status. The installed Linux integrity manifest has SHA-256 eb12e2bce3baf8ee73f426b162c67d0ef39548c7b062c8e4f19a8a2124233f72. The Windows installer and its protected manifest were validated in the dedicated Windows release cycle; do not attribute the Linux manifest hash to Windows.

This protection is independent of the scientific calibration manifests, which document the integrity of experimental artifacts and their own hashes.

Chapter 8

Requirements, installation and usage

8.1 Software dependencies, libraries, services, and requirements

SFAAO is a predominantly local desktop application. Case management, persistence, inference with models already available, calibration with local data, comparison, and report generation do not depend on a commercial AI API or a mandatory cloud service.

8.1.1 Python platform and runtime

A **runtime** is the execution environment required for the application. The validated Windows 2.4.0 installer includes a private Python runtime, so ordinary end-user operation **does not depend on a globally installed Python**. Exact versions must be inspected in the distributed runtime rather than inferred from a preliminary specification.

Development-time version reference (verify installed runtime versions before treating these as definitive):

```
Python 3.10.11
PyTorch 2.12.0+cpu
torchvision 0.27.0+cpu
CUDA = False
```

The distributed Windows 2.4.0 release is **CPU-only** and uses a validated private runtime. **CUDA** is NVIDIA's GPU-computing platform; the Windows CPU installer does not require an NVIDIA GPU.

8.1.2 Main libraries

Dependency	Function in SFAAO
PyQt5	Desktop graphical interface.
PyTorch	Neural networks, training, and inference.
torchvision	Backbones, pretrained weights, and transformations.
NumPy	Numerical computation.
OpenCV	Image processing when used by the pipeline.
Matplotlib	Scientific plots.
SQLite / sqlite3	Local transactional persistence.
python-docx	DOCX document generation.
ReportLab	PDF generation/support when used by the pipeline.
cryptography	Cryptographic licensing and security operations.
Pillow	Image manipulation.
scikit-learn	Auxiliary evaluation functions when used.

The Windows 2.4.0 installer includes its private Python runtime; exact dependency versions must be verified against the release-specific validated build manifest.

8.1.3 Internet and pretrained weights

Normal SFAAO operation does not require a continuous Internet connection. When `pretrained=true`, `torchvision` may need to download the backbone checkpoint on first use if that weight is not already present in the local cache. This download is not remote inference: once obtained, the model runs locally.

```
pretrained backbone requested
 ↓
checkpoint available locally?
 ├─ YES → load locally
 └─ NO → initial download → local cache
```

External datasets must be obtained by the user/institution directly from their legitimate sources and in accordance with their respective licenses.

8.1.4 Operating-system requirements

8.1.4.1 Linux

The Linux version uses restrictive POSIX permissions and requires an environment compatible with the distributed libraries. The interface was correctly validated in the Linux environment used during development.

8.1.4.2 Windows CPU — release 2.4.0 requirement

The Windows 2.4.0 CPU release targets Windows 11 x64 and includes a private Python runtime; program files and persistent data are separated under Program Files and ProgramData.

8.1.5 Hardware

SFAAO can operate entirely on CPU. Training deep models may be faster in environments with a compatible GPU; the Windows CPU distribution described in this manual does not depend on an NVIDIA GPU.

Memory consumption and execution time depend mainly on the backbone, batch size, number of pairs/triplets/images, dataset size, number of epochs, and preprocessing.

Among the validated ResNet backbones, ResNet50 has a higher computational cost than ResNet18 and ResNet34. Official TorchVision documentation reports 25,557,032 parameters and 4.09 GFLOPS for the reference ResNet50 [17]. Therefore, with hardware, resolution, batch size, and other conditions kept comparable, **any SFAAO training mode that uses ResNet50 — PAIR, TRIPLET, or CLASSIFICATION — tends to require more processing time**.

This behavior does not define an absolute duration: execution time depends on CPU/GPU, number of images/pairs/triplets, batch size, number of epochs, `freeze_backbone`, I/O, workers, and other parameters. The Windows 2.4.0 cycle included functional training tests; timing and metrics depend on the configuration and must be recorded for each run.

8.1.6 Windows CPU installation

The Windows 11 x64 CPU distribution of SFAAO 2.4.0 has an Inno Setup installer validated in the test environment, including its dedicated `SFAAO.exe` launcher and private Python runtime. Program binaries reside under `C:\Program Files\SFAAO`; persistent data is stored under `C:\ProgramData\SFAAO`. Installer junctions preserve `storage`, `models/default`, `models/custom`, and `security/state` during upgrade/reinstallation, according to the Windows 11 validation cycle [18], [19].

The release installer is `SFAAO_Setup_2.4.0_CPU_FINAL_PRO_LAUNCHER_RC1.exe`; its SHA-256 is recorded in the distribution section. Preservation of existing cases, models, and settings and successful

GUI startup were validated on Windows 11. Distributed Python modules are PyArmor-protected; the administrative license issuer and private signing key are **not** included in the client.

Activate the client using the public `sfaao-license-request.cmd` and `sfaao-license-install.cmd` tools: send the generated license request to an authorized issuer, receive `license.sfaao`, and install it in persistent client storage. Do not attach private keys, manufacturer administrative tools, or credentials to forensic reports. This manual makes no assertion of completed Authenticode code signing without specific verification evidence.

8.1.7 Linux 2.4.0 CPU installation

The final release has two independent Linux packages.

8.1.7.1 Debian, Ubuntu, and Linux Mint

Debian documentation supports installing a local `.deb` file with `apt install /path/package.deb`, allowing dependencies available in configured repositories to be resolved [20]. For SFAAO:

```
sudo apt install ./sfaao_2.4.0_amd64.deb
sfaao-license-request
sfaao-license-install /path/license.sfaao
sfaao
```

8.1.7.2 Rocky Linux 9 / RHEL-compatible

Red Hat documentation recommends installing a local RPM with `dnf install <path_to_RPM_file>` [21]. For SFAAO:

```
sudo dnf install ./sfaao-2.4.0-1.x86_64.rpm
sfaao-license-request
sfaao-license-install /path/license.sfaao
sfaao
```

On Linux, `/usr/bin/sfaao` creates a user-visible `~/SFAAO_Data` shortcut when absent, pointing to `~/.local/share/SFAAO/runtime/data`. Its `datasets`, `images` and `models` directories are mounted inside the container as `/data/datasets`, `/data/images` and `/data/models`, respectively. Persistent operational data remain separate from the protected payload.

The internal `/opt/sfaao/run-sfaao.sh` script allows isolated testing with the `SFAAO_ROOT` environment variable, but does **not** create the visible `~/SFAAO_Data` shortcut; only `/usr/bin/sfaao` handles that step. Do not redirect the production shortcut to a temporary validation directory.

On the validated Rocky Linux 9.8 environment, SELinux remained **Enforcing**. The launcher detects SELinux and applies private `:Z` labeling to volumes controlled by SFAAO; during license installation, the received file is mounted read-only with `:ro,Z` when SELinux is active. Podman documentation describes `Z` as private relabeling and `ro` as a read-only mount [22].

8.1.8 Protection of distributed code — PyArmor Pro

The Linux 2.4.0 payload uses **PyArmor Pro 9.2.6**, with runtime `pyarmor_runtime_015424` [23]. The PyArmor license reports BCC and RFT capability, but licensed capability is not treated as evidence that either mode is enabled in the distributed payload: effective build status must be established from the corresponding build script/log. Any change to those modes requires a new build and validation cycle. This protection increases resistance to inspection and direct reuse of distributed Python code, without being presented as an absolute guarantee against reverse engineering.

PyArmor is a code-protection layer and does not replace SFAAO's own operational licensing, integrity verification, authentication, role-based authorization, or traceability. The private PyArmor regfile/regcode, the SFAAO licensing private key, and the administrative issuer do not belong on the client and must never be distributed.

When `pretrained=true`, TorchVision may download weights on first use if the checkpoint is not already cached [17]. The Linux 2.4.0 payload includes the public certificate required by the application's TLS/CA path. Functional training validation of the installed 2.4.0 release must record whether weights were downloaded, reused from cache, or obtained without network access; the Windows cycle must repeat this verification separately.

8.2 Usage: adding a dataset

```
ADMIN
 → Manage Datasets
 → Add Dataset
 → name
 → profile
 → version
 → directory
 → audit 0–100%
 → statistics
 → registration
```

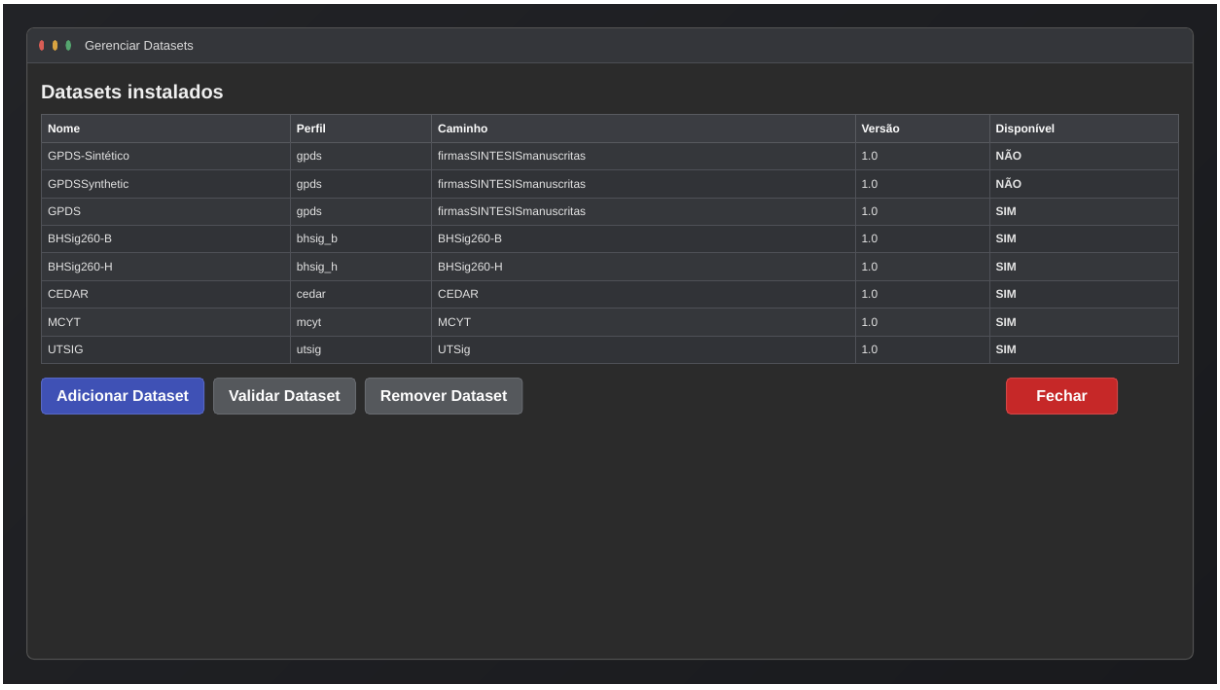


Figure 8.1: Installed Dataset Manager with availability listing and add, validate, and remove actions.

8.3 Usage: models

A model may be:

- trained by SFAAO;
- imported as a compatible package.

The ADMIN selects the active operational model. Historical models remain linked to results already produced, even when another package becomes active.

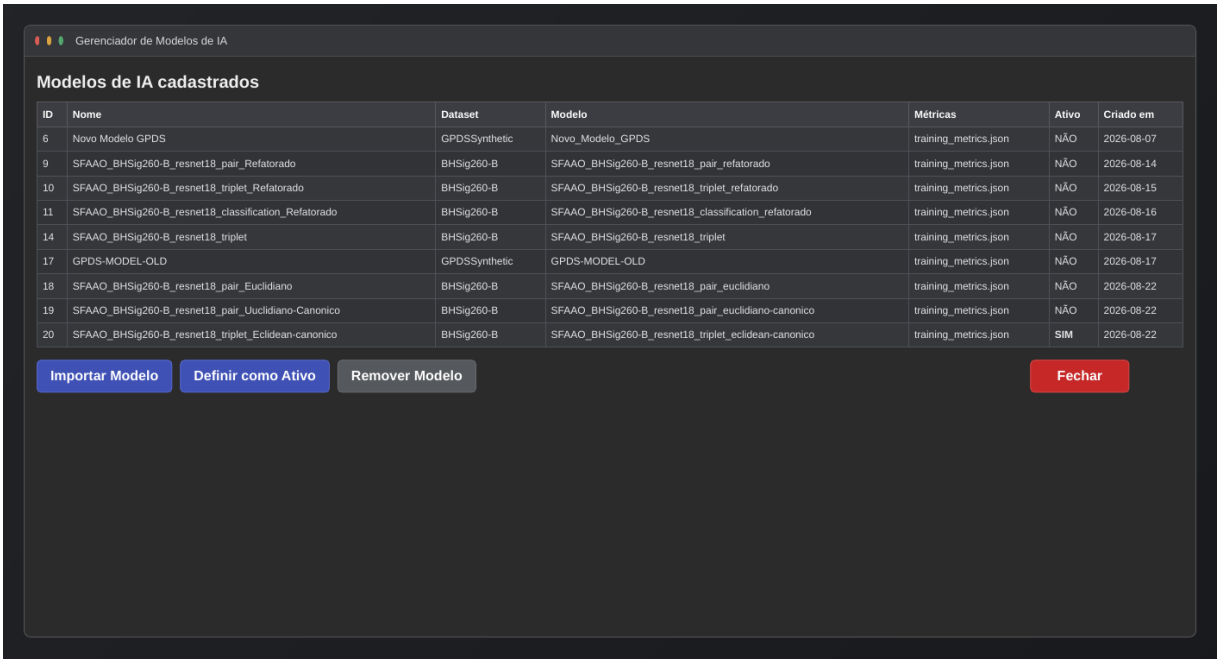


Figure 8.2: AI Model Manager showing the registered model table and import, obtain, and remove actions.

8.4 Usage: training

The ADMIN configures:

- dataset;
- backbone;
- loss;
- metric;
- embedding;
- dropout;
- pretrained;
- epochs;
- batch;
- learning rate;
- optimizer;
- scheduler;
- sampling;
- early stopping.

The resulting package is registered in the Model Manager. The final framework provides PAIR, TRIPLET, and CLASSIFICATION; ResNet18/34/50, MobileNetV3, EfficientNet-B0, and ConvNeXt-Tiny; Contrastive, Cosine Contrastive, Triplet, and ArcFace losses according to mode; Cosine and Euclidean distances; Adam/AdamW; and none, StepLR, cosine, and plateau schedulers.

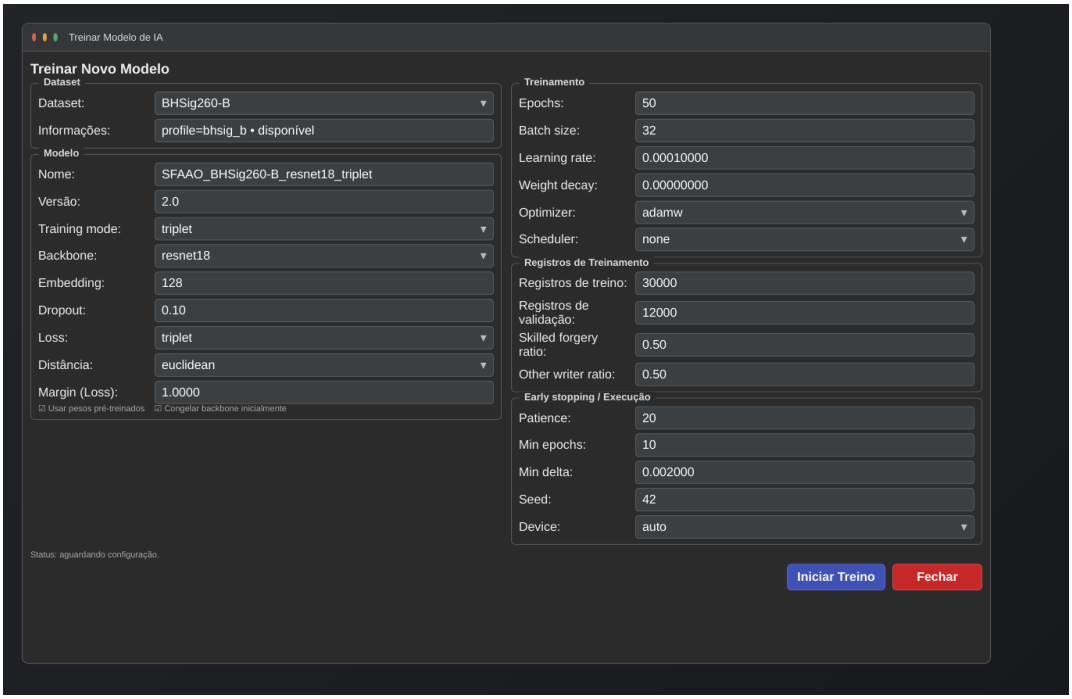


Figure 8.3: Training setup form with dataset, architecture, mode, optimization parameters, and execution settings.

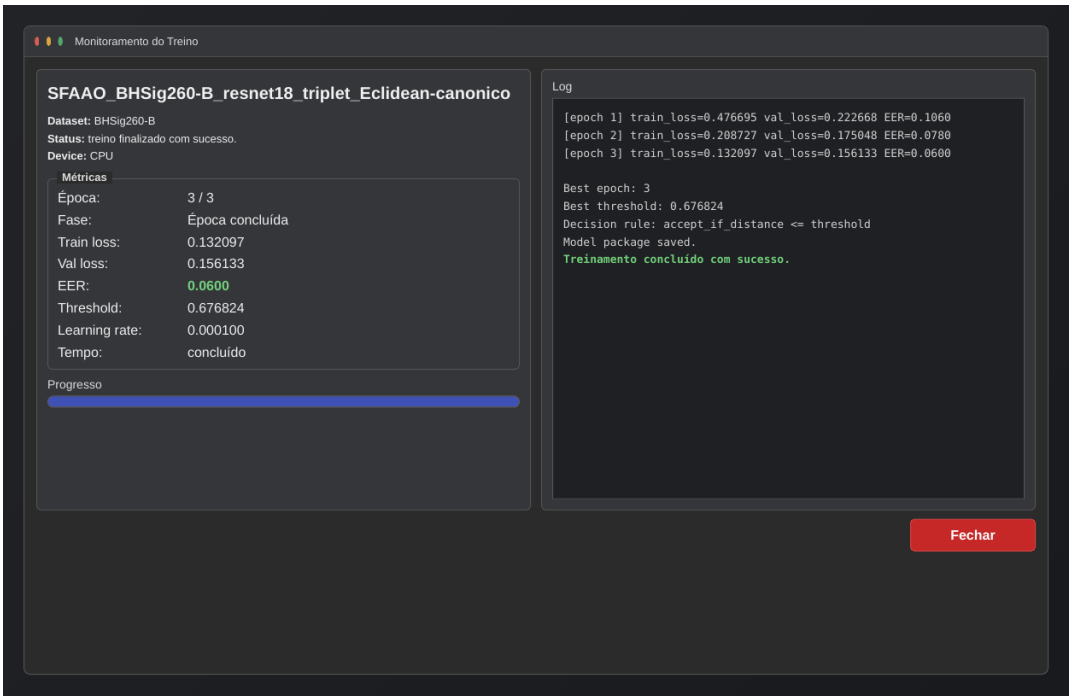


Figure 8.4: Training monitor with execution progress, metrics, and log messages.

8.5 Usage: calibration

Before comparison:

```
case
 → model
 → RC/RO
```

```
→ protocol
→ parameters
→ execute
→ review metrics
→ select calibration
```

Opening the calibration window in the final release uses an indeterminate progress dialog and creates the window on Qt's main thread, reducing the perception of freezing without moving widgets to secondary threads.

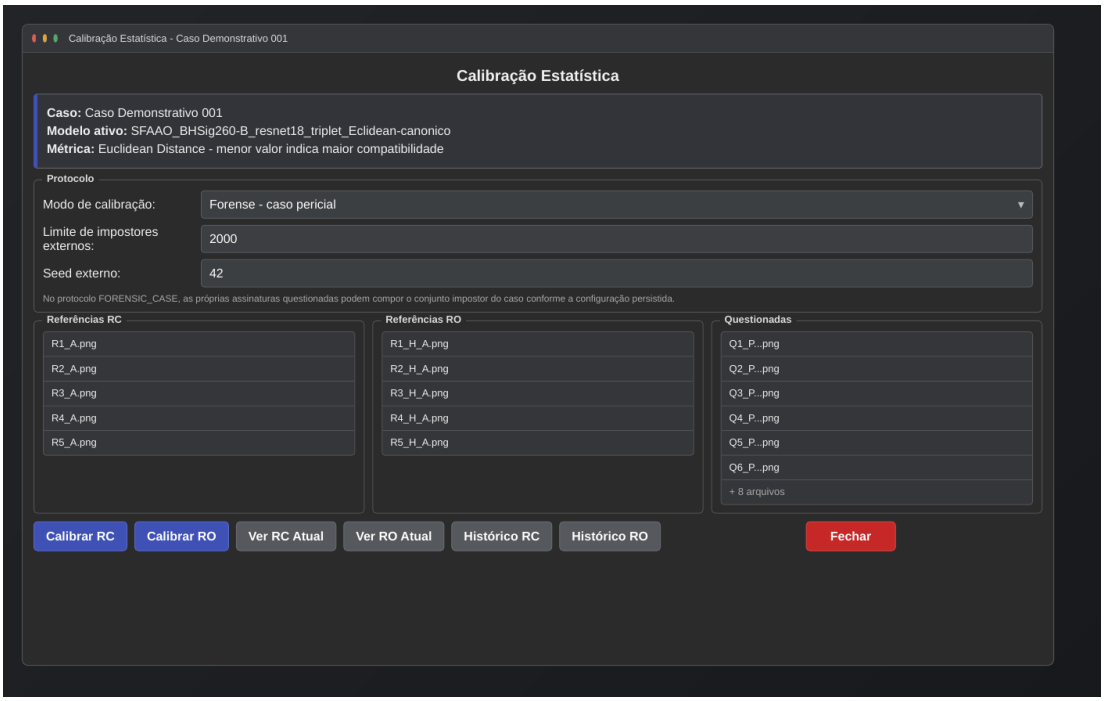


Figure 8.5: Statistical calibration screen in forensic mode with active model, distance metric, parameters, and RC, RO, and questioned sample sets.

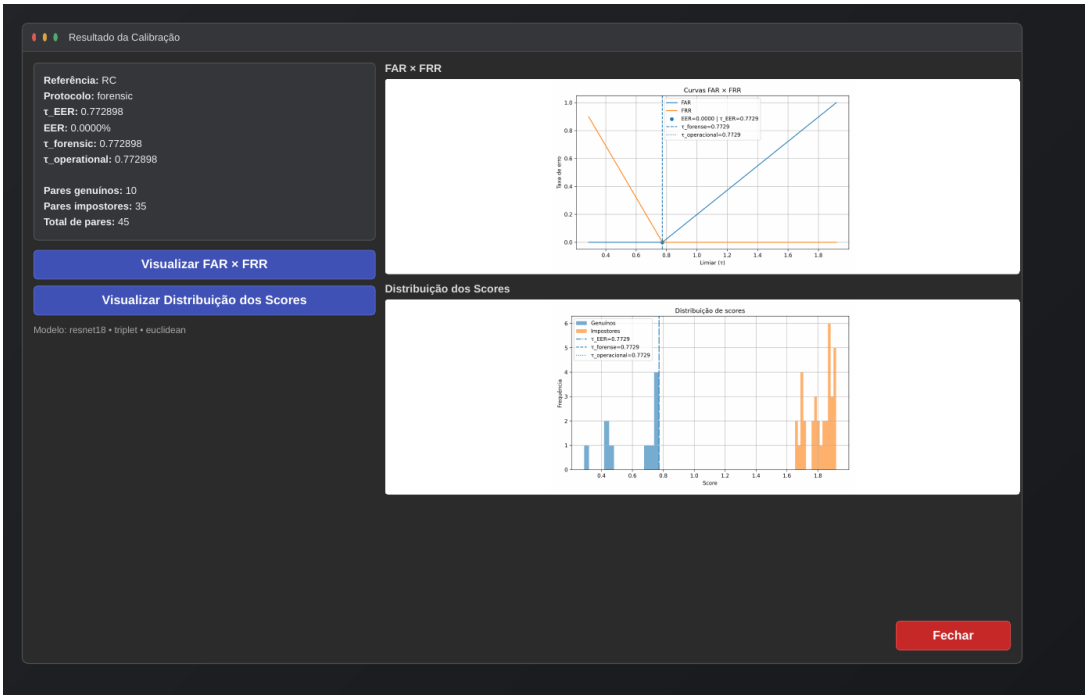


Figure 8.6: Calibration results for RC reference, showing EER, thresholds, pair counts, FAR/FRR curves, and score distribution.

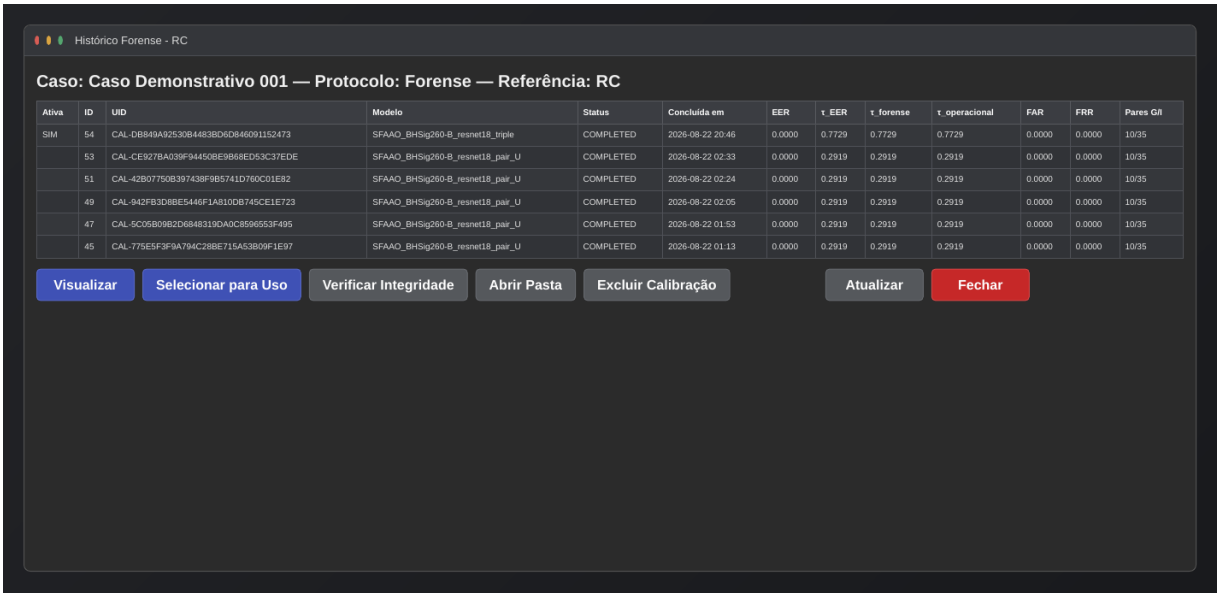


Figure 8.7: Case calibration history, including traceable records, result selection, and visualization actions.

8.6 Usage: comparison

In release 2.4.0, the **Signature Comparison** entry opens a four-option window: **(1) Individual Comparison (LORO/A4)**, **(2) Inspect Pairwise Score**, **(3) Batch Comparison**, and **(4) Refresh Context**. These actions distinguish an operational decision, elementary-score inspection, batch processing, and refresh of selected case context.

8.6.1 Individual Comparison (LORO/A4)

Choose the case, active model, and valid calibration; select **RC** or **RO** references and a **QUESTIONED** signature. The comparison uses the active context and requires a match between the metric, statistical

calibration domain, and inference. Under aligned LORO, each $A(N-1)$ fold is a decision unit; elementary **A1** is provenance information, while $A(N)$ is descriptive only and must not be compared directly to a LORO threshold.

8.6.2 Inspect Pairwise Score

Inspect an individual score between two signatures for diagnostic and provenance purposes. **An A1 score is not, by itself, the LORO/A4 operational outcome**, and it does not replace fold aggregation or the calibration decision policy.

8.6.3 Batch Comparison

Process eligible comparisons within the case context and review persisted results, with fold-level details where applicable. CSV export remains subject to license entitlements and quotas. Review references, score domains, threshold, stability, and stored provenance before including results in a report.

8.6.4 Refresh Context

Reload the comparison window's case/model/calibration context after changes to the active case, references, model, or calibration. Refreshing context does not automatically perform new inference or recalibration.

8.6.5 Threshold direction and interpretation

For **Euclidean Distance**, smaller values indicate greater compatibility and acceptance uses $d \leq \tau$. For **Cosine Similarity**, larger values indicate greater compatibility and acceptance uses $s \geq \tau$. Only apply thresholds matching the configured protocol and decision statistic. Computational outcomes do not automatically classify the authenticity of QUESTIONED signatures.

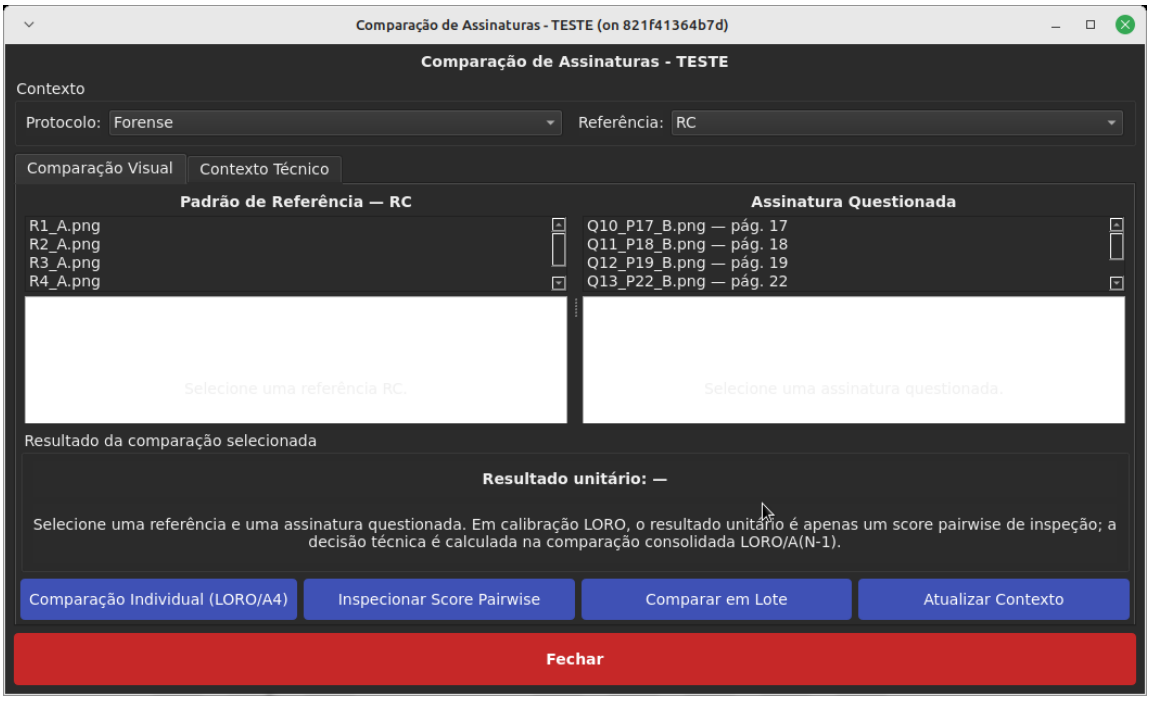


Figure 8.8: Signature Comparison 2.4.0: active context, RC/RO references, questioned samples, and four available actions.

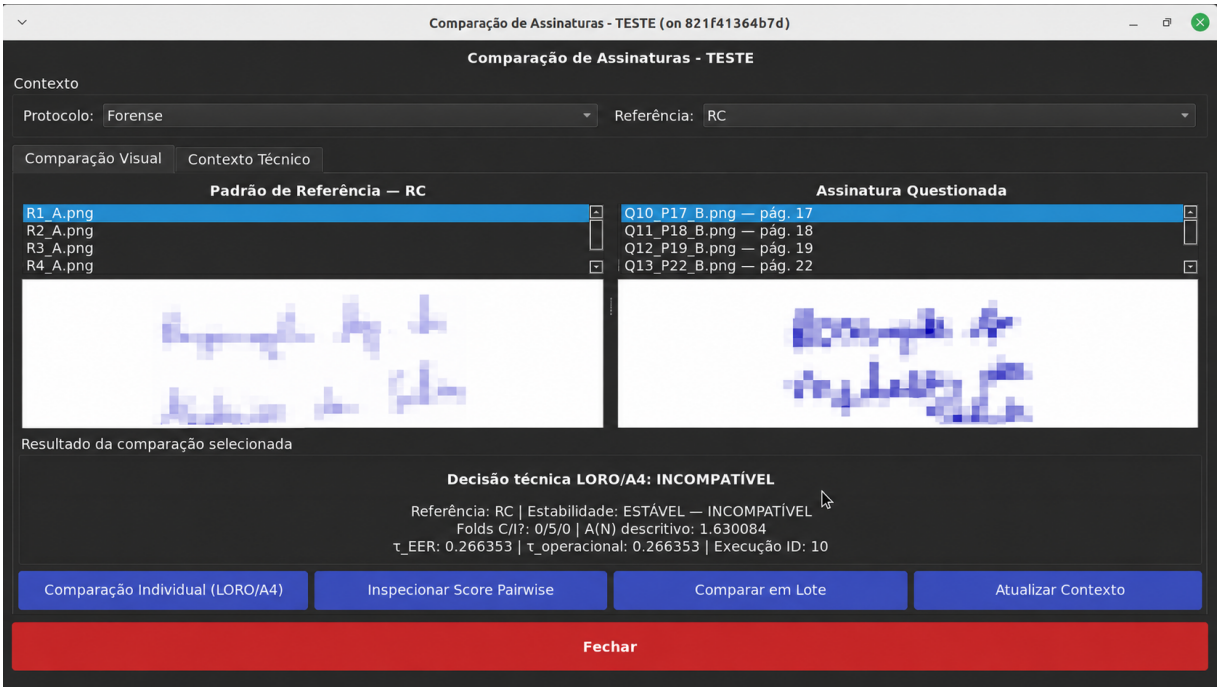


Figure 8.9: Individual LORO/A4 comparison using RC references: selected samples, stability, and the technical decision displayed by the application.

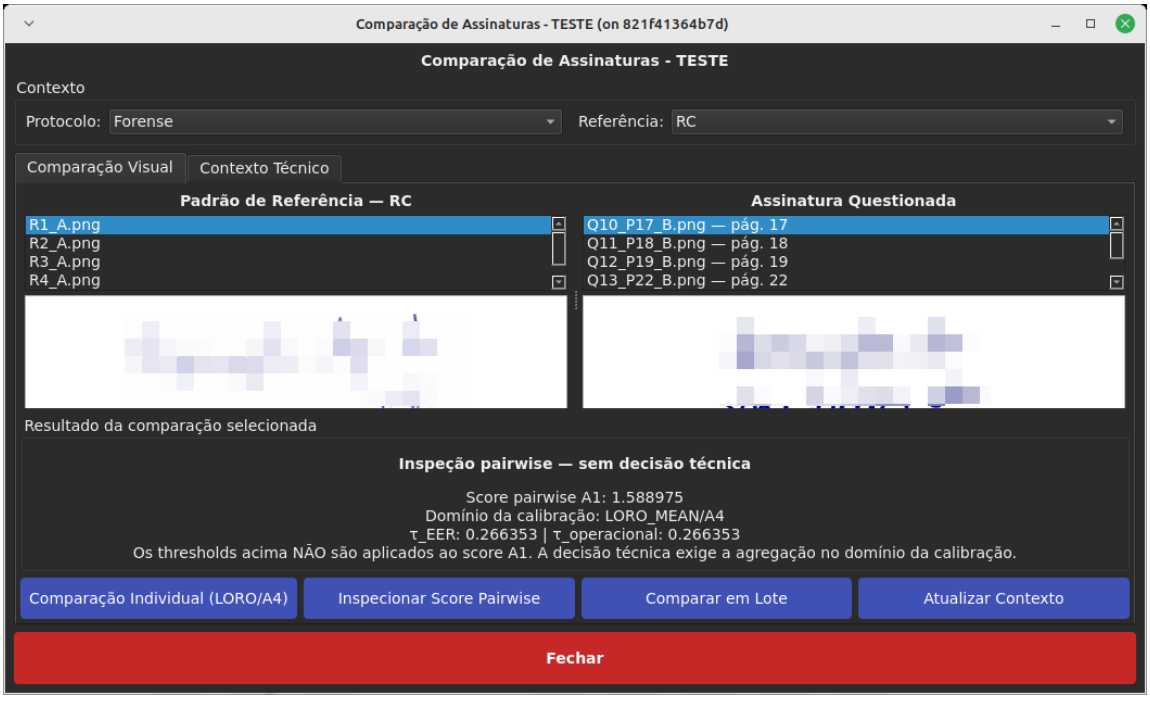


Figure 8.10: Pairwise A1 score inspection: single-pair assessment without a technical decision; calibration thresholds are not applied directly to this score.

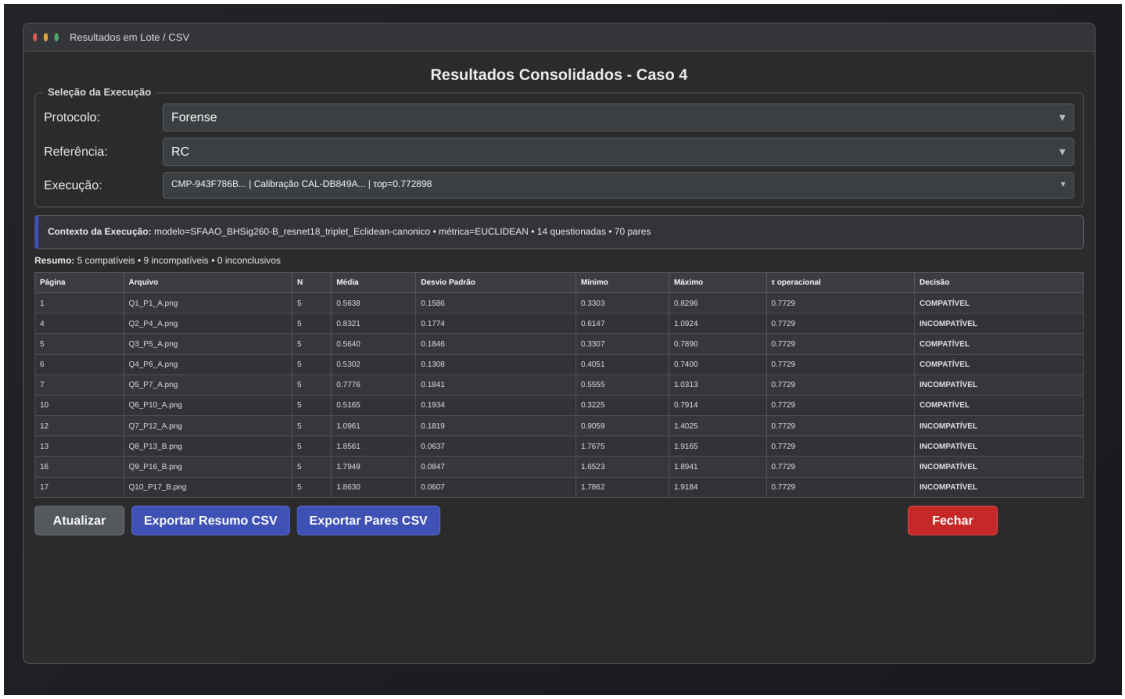


Figure 8.11: Consolidated batch-comparison results table, with filters and CSV export options.

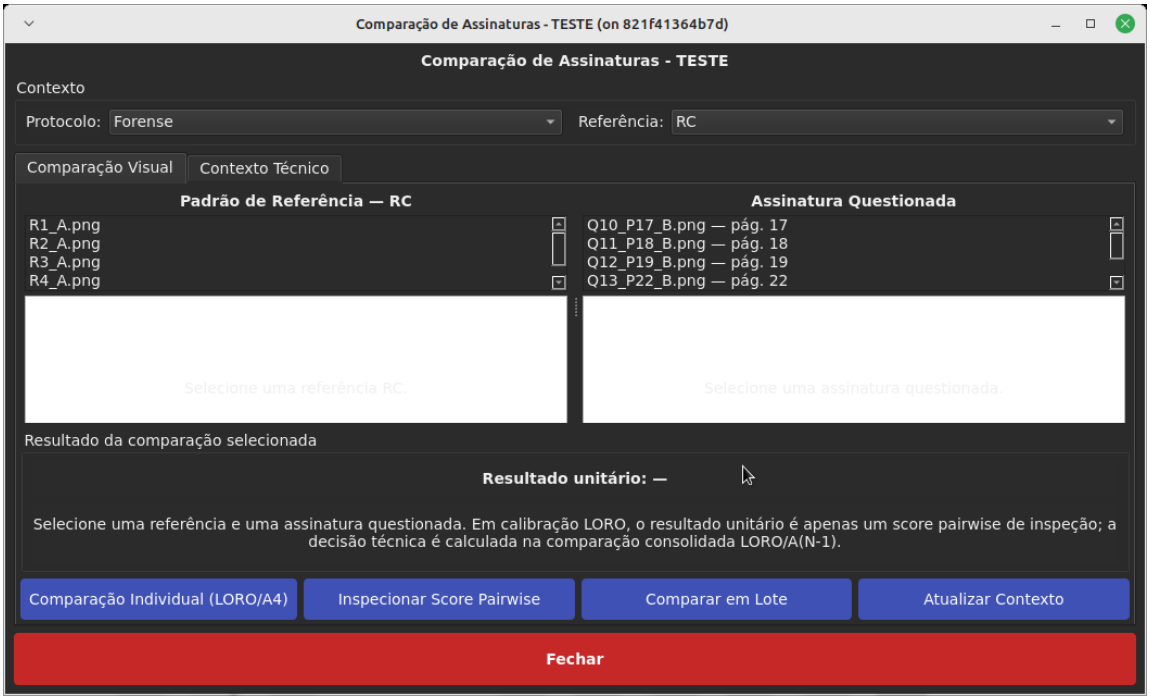


Figure 8.12: Comparison window after Refresh Context: selection and individual result reset without automatically running another comparison.

8.7 Usage: report

Reports must follow review of the case, model, calibration, comparisons, and the examiner's professional details. The qualified examiner is responsible for the handwriting conclusion and must review every document before release.

DOCX generation validated on all three SFAAO 2.4.0 platforms: DOCX report generation passed functional validation on Windows 11, Linux Mint, and Red Hat, as confirmed by the technical owner.

On Linux Mint, the isolated validation environment ran the image embedded in the October 19, 2026 packages, with a verified image TAR hash. On Windows, DOCX generation was tested with the dedicated launcher in the 2.4.0 installer. On Red Hat, functional validation included the RPM package and DOCX generation. These results do not replace review of document formatting, forensic content, evidence integrity, or the examiner's signature.

Where supported, DOCX and PDF are complementary formats of the same logical report, retaining identity and quota semantics described in the reporting sections. Producing a second format is not a new forensic conclusion. Record the case, model, run, calibration, and report version identifiers.

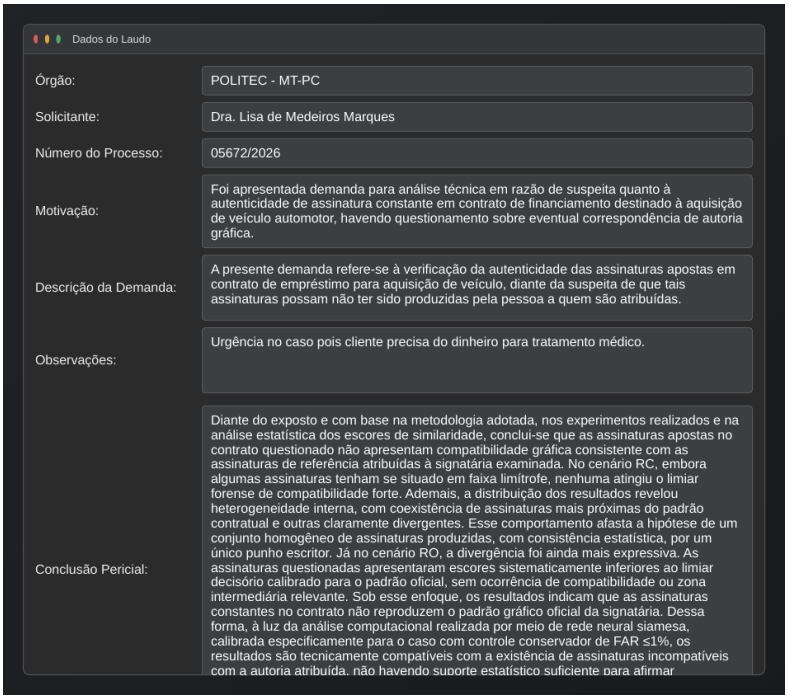


Figure 8.13: Forensic report metadata form with identification, rationale, and conclusion fields.

The next figure shows the forensic report in **DOCX** format, opened in LibreOffice Writer. The editable format supports human review of the document content and presentation before final issue. The subsequent image separately shows the PDF report.

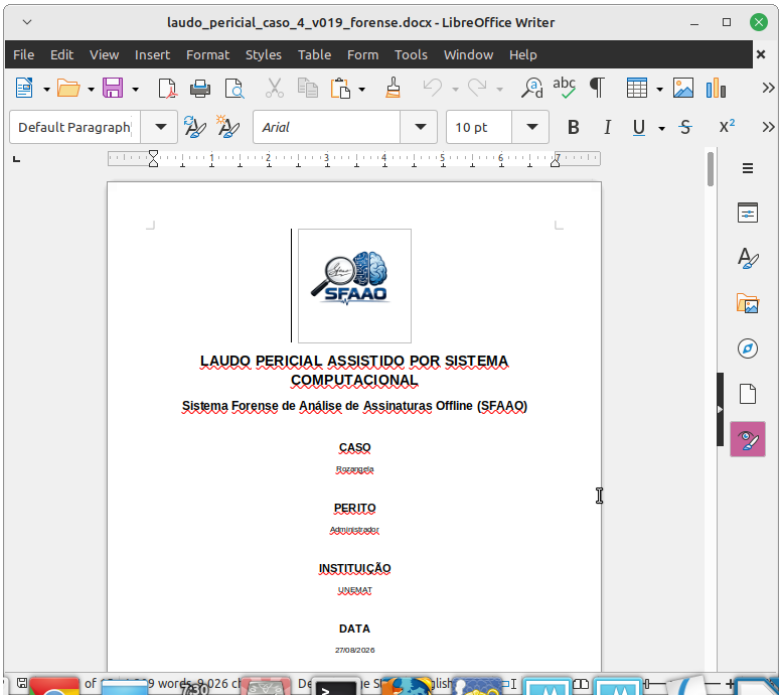


Figure 8.14: Computer-assisted forensic report in DOCX format, opened in LibreOffice Writer. The title bar identifies the file with the `.docx` extension; the page displays the case, examiner, and institution information.

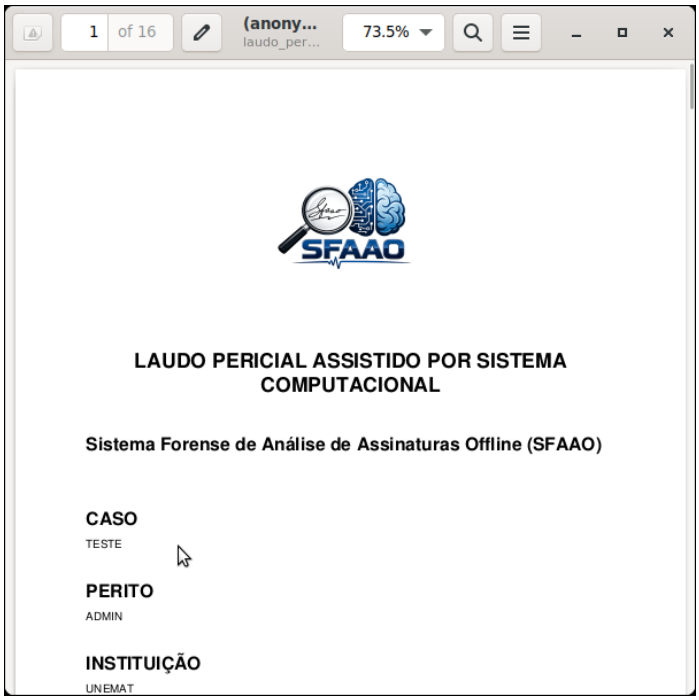


Figure 8.15: First page of an SFAAO 2.4.0 forensic report displayed in a PDF viewer, showing the case, examiner, and institution. This figure illustrates the PDF report, not a DOCX document.

Chapter 9

Governance, distribution and validation

9.1 Dataset policy and acquisition responsibility

9.1.1 Datasets are not part of the commercial product

SFAAO is distributed **without third-party signature datasets**.

Acquisition/licensing of SFAAO software **does not include**:

```
GPDS
CEDAR
BHSig260
UTSig
MCYT
or any other third-party dataset
```

The commercial product contains the software, integration mechanisms, and, where applicable, **profiles/adapters** capable of interpreting specific dataset structures.

Dataset content remains the property and responsibility of its respective rights holders.

9.1.2 Responsibility of the user or institution

To train a model or use an external dataset in scientific calibration, the user/institution must:

```
1. identify the desired dataset
2. review the license terms
3. obtain authorization when required
4. download it directly from the legitimate source
5. store the dataset locally
6. select the corresponding profile in SFAAO
7. validate the dataset structure
8. use the dataset only within the rights granted
```

SFAAO **does not bypass dataset access, authentication, or licensing mechanisms**.

9.1.3 Support does not mean redistribution

When the manual states that SFAAO “supports GPDS”, “supports CEDAR”, or “supports BHSig260”, this means:

> the architecture has, or may have, a profile capable of interpreting the structure of the dataset legitimately obtained by the user.

It does not mean that:

- the dataset is embedded in the installer;
- the SFAAO license grants rights over the dataset;
- the dataset is sold together with the software;
- SFAAO redistributes images from those datasets.

9.1.4 License status

Terms of use vary among datasets and may change. Therefore, before commercial or institutional use, the current terms at the official source must be checked.

As a conservative distribution policy for SFAAO 2.4.0:

```
NO THIRD-PARTY DATASET IS REDISTRIBUTED
```

This policy applies regardless of whether the profile is present in the installation.

9.1.5 Dataset for training

Training:

```
legitimately obtained dataset
 ↓
local storage by the institution
 ↓
registration in SFAAO
 ↓
validation
 ↓
pair/triplet generation
 ↓
training
```

9.1.6 Dataset for SCIENTIFIC_EXTERNAL

Scientific calibration:

```
case
 ├─ RC
 ├─ RO
 └─ QUESTIONED
       │
       │  QUESTIONED reserved for inference
       │
       ▼
legitimately obtained external dataset
 ↓
external-impostor selection
 ↓
RC or RO calibration
```

The external dataset does not replace RC/RO and does not incorporate questioned signatures into the calibration population.

9.1.7 Institution-owned datasets

An institution may use:

- its own dataset;
- its own synthetic dataset;

- an institutional collection;
- an authorized third-party dataset.

For new datasets, a compatible profile/adapter must be created or configured, preserving classification, writer IDs, and traceability.

9.1.8 Privacy, chain of custody, and rights

Before importing institution-owned datasets, the institution is responsible for verifying:

- legal basis for data processing;
- usage rights;
- purpose;
- retention policy;
- anonymization/pseudonymization where applicable;
- chain of custody;
- personal-data protection.

SFAAO provides technical mechanisms for analysis and storage; legal compliance of an imported dataset depends on the institutional context.

9.2 Model policy

The initial commercial release is distributed:

```
WITHOUT TRAINED MODELS
```

The institution trains its own model or imports a model whose use is authorized.

Models used during validation are not included in the installer.

9.3 Distribution

The SFAAO 2.4.0 CPU release is organized under `SFAAO_DISTRIBUTION_2.4.0`. The commercial software package does not include third-party datasets, validation models, real cases, or manufacturer-private administrative utilities.

- **DEB (amd64):** `01_DEB/sfaao_2.4.0_amd64.deb`. SHA-256: `5e7c730ed0cdd59426d896f31dc8a277ef897fba104aa38511cec0e3abc522a5`.
- **RPM (x86_64):** `02_RPM/sfaao-2.4.0-1.x86_64.rpm`. SHA-256: `d5ead5e0fe03e2fb10fef602afc93200ff568398cda20745dc237ebfb8de4d4e`.
- **Windows CPU:** `03_WINDOWS_EXE/SFAAO_Setup_2.4.0_CPU_FINAL_PRO_LAUNCHER_RC1.exe`. SHA-256: `5de43c5dba6559e55442e4c25c6ba13303c95561aef09d2630059b160f78c66d`.

The single distribution-level checksum manifest is `06_CHECKSUMS/SHA256SUMS`. From the distribution root, validate packages using `sha256sum -c 06_CHECKSUMS/SHA256SUMS` on Linux, or compare corresponding calculated hashes on Windows. All three distributed installer copies passed verification on October 27, 2026. The OCI image SHA-256 common to both Linux installers is `e2d04f319fb57ff712af26deb5efe666bb2763d26495f287e53bd9c029dba362`.

The RPM structure, payload, digest, and embedded image were verified; **installation and functional operation on Red Hat were also validated**, including DOCX report generation, as confirmed by the technical owner. This validation applies to the tested environment and must not automatically be extended to every Fedora, Rocky, or AlmaLinux version.

9.3.1 What the buyer receives

The buyer/institution receives the official platform package, runtime, launchers, client license-request/license-install tools, authorized profiles/adapters, training, inference, calibration, comparison, reporting, security, integrity, licensing mechanisms, and the public documentation required for authorized operation.

9.3.2 What the buyer does not receive

The client does not receive third-party datasets, third-party biometric images, validation models, real cases, development databases, private caches, the private key, the license issuer, or the private PyArmor regfile. Acquisition of SFAAO does not automatically transfer rights over third-party datasets.

9.3.3 Flow after acquisition

```
obtain official package + verify SHA-256
 -> install SFAAO
 -> run sfaao-license-request
 -> send SFAAO_LICENSE_REQUEST.json to the authorized issuer
 -> receive license.sfaao
 -> run sfaao-license-install
 -> create the first ADMIN
 -> legitimately obtain a dataset
 -> register/validate the dataset
 -> train an institution-owned model or import an authorized model
 -> operate cases and protocols permitted by the license
```

9.4 Testing and validation

In October 2026, the SFAAO 2.4.0 CPU DEB, RPM, and Windows installers were assembled in the official distribution and their SHA-256 hashes checked. The Linux packages, built on October 19 after the DOCX fix, contain matching manifests and an identical OCI image, verified directly inside each installer.

9.4.1 Linux Mint, DEB, and Podman

The installed `/opt/sfaao` payload passed SHA-256 verification for its OCI image, installation script, startup script, and internal integrity manifest. The launcher prefers Podman, forwards the GUI over X11, and maintains persistent data under `~/.local/share/SFAAO`. The host link `~/SFAAO_Data` points to the main data folder, exposing `datasets`, `images`, and `models`.

A separate `SFAAO_HOMOLOG_DOCX_2.4.0` environment was used with independent data, a test license, and the image matching the installers. The technical owner confirmed application operation and **successful DOCX report generation**. Release-level functional validation was also confirmed on Windows 11 and Red Hat; this isolated session record does not replace platform-specific scientific test records.

9.4.2 Windows 11 x64 CPU

The 2.4.0 Windows installer with dedicated launcher was built and tested on Windows 11, including the protected-payload integrity check, upgrade over an earlier version with persistent data preserved, and functional operation. **DOCX report generation was also functionally validated on Windows 11**, as confirmed by the operator. Windows protected modules were audited. The release EXE SHA-256 matches the validated artifact.

9.4.3 Red Hat (RPM): validated installation and operation

The SFAAO 2.4.0 RPM passed structural, digest, manifest, and embedded-image hash checks. The technical owner also confirmed **successful installation and functional operation on Red Hat, including DOCX report generation**. The precise Red Hat version and detailed test record were not supplied for this editorial revision; this confirmation does not automatically certify every compatible RPM-based distribution.

9.4.4 Release evidence policy

Validation is bound to a specific artifact and test environment. Changes to code, assets, runtime, client tools, or installer require impact analysis, new hashes, and proportional testing. Third-party biometric data, private keys, and license issuers remain outside the commercial release.

9.5 Limitations

- image quality affects performance;
- generalization depends on the dataset;
- domain shift may alter distributions;
- AI is probabilistic;
- thresholds require context;
- calibration is specific to the model and case;
- the number/representativeness of references is relevant;
- SFAAO does not replace forensic handwriting examination.

Chapter 10

Best practices and detailed technical architecture

10.1 Best practices

- preserve original images;
- document the origin of references;
- use the correct case;
- verify the active model;
- calibrate before comparing;
- do not reuse calibration from another model;
- do not alter evidence after comparison;
- preserve artifacts;
- review UIDs;
- review the report;
- use a consistent backup;
- for SQLite in WAL mode, prefer the SQLite Backup API instead of raw copying of an open database.

10.2 Extensibility

The platform can receive:

- new backbones;
- new losses;
- new metrics;
- new optimizers;
- new schedulers;
- new profiles;
- new sampling strategies;
- new calibrations;
- new reports;
- future embedding-based biometric modalities.

10.3 Recommended structure of the private repository

```
README.md
CHANGELOG.md
CONTRIBUTING.md
SECURITY.md
LICENSE
docs/
  ARCHITECTURE.md
  INSTALLATION.md
  USER_MANUAL.md
  TUTORIAL.md
  DATASETS.md
  SECURITY.md
  TRACEABILITY.md
```

```
  GLOSSARY.md
app.py
ai/
controllers/
database/
reports/
security/
storage_services/
ui/
scripts/
tests/
requirements.txt
```

10.3.1 Implementation map: modules, files, and classes

This section relates the conceptual SFAAO architecture to the main components of the 2.4.0 implementation confirmed in the codebase reviewed during validation. Its purpose is not to replace source-code documentation, but to allow maintainers to quickly identify where each responsibility is located.

10.3.1.1 Startup, authentication, and administration

- **Application entry point** — `app.py` — runtime initialization
- **First administrator** — `controllers/first_admin_controller.py` — `FirstAdminController`
- **First administrator - GUI** — `ui/first_admin_window.py` — `FirstAdminWindow`
- **Login** — `ui/login_window.py` — `LoginWindow`
- **Users** — `database/repositories/user_repository.py` — `UserRepository`
- **User administration** — `controllers/admin_user_controller.py` — `AdminUserController`
- **User administration - GUI** — `ui/user_manager_window.py` — `UserManagerWindow`
- **Cryptographic policy** — `security/auth.py` — authentication/hash functions
- **Storage permissions** — `security/storage_permissions.py` — POSIX/Windows protection
- **Main window** — `ui/main_window.py` — `MainWindow`

10.3.1.2 Cases, signatures, and forensic operation

- **Case operations** — `ui/case_operations_window.py` — `CaseOperationsWindow`
- **Case metadata** — `ui/case_metadata_window.py` — `CaseMetadataWindow`
- **Examiner profile** — `ui/expert_profile_window.py` — `ExpertProfileWindow`
- **Signature persistence** `database/repositories/signature_repository.py` `SignatureRepository`
- **Image viewing** — `ui/image_viewer_window.py` — `ImageViewerWindow`
- **Forensic-role audit** — `signature_role_audit` — table and associated operations

10.3.1.3 Datasets

- **Dataset reference** — `ai/datasets/dataset_reference.py` — `DatasetReference`
- **Structural profile** — `ai/datasets/dataset_profile.py` — `DatasetProfile`
- **Access/validation** — `ai/datasets/dataset_provider.py` — `DatasetProvider`
- **Catalog** — `ai/datasets/dataset_catalog.py` — `DatasetCatalog`
- **Factory** — `ai/datasets/dataset_factory.py` — dataset factory
- **Loader** — `ai/datasets/dataset_loader.py` — sample loaders
- **Triplet dataset** — `ai/datasets/signature_triplet_dataset.py` — TRIPLET dataset
- **Classification dataset** — `ai/datasets/signature_classification_dataset.py` — CLASSIFICATION dataset
- **Controller** — `controllers/dataset_controller.py` — `DatasetController`

- **GUI** — `ui/dataset_manager_window.py` — `DatasetManagerWindow`

Profiles under `ai/datasets/profiles/` describe supported dataset structures without redistributing the datasets themselves.

10.3.1.4 Configurations

- **Model** — `ai/config/model_config.py` — `ModelConfig`
- **Training** — `ai/config/training_config.py` — `TrainingConfig`
- **Calibration** — `ai/config/calibration_config.py` — `CalibrationConfig`

These configurations form the reproducible identity of the experiment and must not be treated as simple visual preferences.

10.3.1.5 Training

- **Main pipeline** — `ai/training/training_pipeline.py` — `TrainingPipeline`
- **Base pipeline** — `ai/training/base_training_pipeline.py` — training abstraction
- **Capabilities/modes** — `ai/training/training_capabilities.py` — `TrainingCapabilities`
- **Preprocessing/utilities** — `ai/training/utils.py` — auxiliary functions
- **Training interface** — `ui/training_window.py` — `TrainingWindow`
- **Monitoring** — `ui/training_monitor_window.py` — `TrainingMonitorWindow`

Training uses factories to select backbone, loss, optimizer, scheduler, distance, and model according to the configuration.

10.3.1.6 Backbones and model construction

The `ai/backbones/` directory contains feature extractors. Components confirmed/operational in SFAAO 2.4.0 include:

```
ResNet18
ResNet34
ResNet50
EfficientNet-B0
ConvNeXt-Tiny
MobileNetV3
```

Files such as `ai/backbones/resnet18.py`, `resnet34.py`, and `resnet50.py` use shared utilities for constructing the extractor. The other families are resolved by the corresponding modular infrastructure of the release.

Creation must occur through factories/configurations, preserving the principle that the GUI does not instantiate scientific architectures directly.

10.3.1.7 Models and repository

- **Model repository** — `ai/models/model_repository.py` — `ModelRepository`
- **Model control** — `controllers/model_controller.py` — `ModelController`
- **Visual manager** — `ui/model_manager_window.py` — `ModelManagerWindow`

The model package must preserve weights and sufficient metadata to reconstruct its configuration.

10.3.1.8 Calibration

- **Calibration base**
  `ai/calibration/base_calibration_pipeline.py`
  `BaseCalibrationPipeline`

- **Forensic**
  `ai/calibration/forensic_calibration_pipeline.py`
  `ForensicCalibrationPipeline`
- **Scientific**
  `ai/calibration/scientific_calibration_pipeline.py`
  `ScientificCalibrationPipeline`
- **Pair construction** — `ai/calibration/pair_builder.py` — `PairBuilder`
- **Statistics** — `ai/calibration/calibration_statistics.py` — FAR/FRR/EER calculations
- **Result** — `ai/calibration/calibration_result.py` — `CalibrationResult`
- **Execution** — `ai/calibration/calibration_execution.py` — `CalibrationExecution`
- **Artifact** — `ai/calibration/calibration_artifact_builder.py` — `CalibrationArtifactBuilder`
- **Artifact repository** — `ai/calibration/calibration_artifact_repository.py` — artifact repository
- **Database repository**
  `database/repositories/calibration_repository.py`
  `CalibrationRepository`
- **GUI** — `ui/calibration_window.py` — `CalibrationWindow`
- **History** — `ui/calibration_history_window.py` — `CalibrationHistoryWindow`
- **Results** — `ui/calibration_results_window.py` — `CalibrationResultsWindow`

10.3.1.9 Inference and comparison

- **Inference** — `ai/inference/inference_pipeline.py` — `InferencePipeline`
- **Comparison** — `ai/comparison/comparison_pipeline.py` — `ComparisonPipeline`
- **Comparison GUI** — `ui/compare_window.py` — `CompareWindow`
- **Batch results** — `ui/batch_results_window.py` — `BatchResultsWindow`
- **Persistence** — `database/repositories/comparison_repository.py` — comparison repository

10.3.1.10 Reports

- **Orchestration** — `controllers/report_controller.py` — `ReportController`
- **Consolidation** — `reports/report_pipeline.py` — report pipeline
- **DOCX** — `reports/docx_report_generator.py` — DOCX generator
- **PDF** — `reports/pdf_report_generator.py` — PDF generator

Reports consume already-persisted data; they must not silently recalculate case science with parameters different from those associated with the recorded execution.

10.3.1.11 Maintenance rule

The map above represents validated release 2.4.0. In future versions, a class-name or location change must update this manual and the changelog, without silently altering the platform's scientific semantics.

10.3.2 Data model and SQLite schema

SFAAO uses SQLite for metadata, traceability, auditing, and relationships. Images, weights, and scientific artifacts remain in the filesystem and are referenced by the database.

The schema has its own versioning; in the 2.4.0 release cycle there are structures internally identified by schema version, such as schema v6. The version number of the **schema** must not be confused with the version of the **scientific protocol** or with the commercial SFAAO version.

10.3.2.1 Core tables

Table	Function
schema_versions	schema versioning/migrations
users	credentials, roles, and user state
expert_profiles	examiner professional data
cases	central forensic-examination unit
case_metadata	descriptive information and human conclusion
signatures	RC, RO, and QUESTIONED
signature_role_audit	history of forensic-classification changes
datasets	local catalog of legitimately obtained datasets
trained_models	catalog of registered models
calibrations	case+model calibration execution/configuration
calibration_samples	samples explicitly registered for calibration when applicable
calibration_results	calibration metrics and artifacts
comparison_runs	identifiable comparison execution
similarity_results	individual reference-questioned scores
questioned_results	aggregation and decision per questioned signature
reports	versions of issued reports

10.3.2.2 Main relationships

```
USERS
  ├── EXPERT_PROFILES
  └── CASES
        ├── CASE_METADATA
        ├── SIGNATURES
        │     └── SIGNATURE_ROLE_AUDIT
        ├── CALIBRATIONS ──── TRAINED_MODELS
        │     ├── CALIBRATION_SAMPLES
        │     └── CALIBRATION_RESULTS
        ├── COMPARISON_RUNS ── TRAINED_MODELS
        │     ├── SIMILARITY_RESULTS
        │     └── QUESTIONED_RESULTS
        └── REPORTS
```

10.3.2.3 users

Functional fields include:

```
id
username
password_hash
full_name
email
role
is_active
failed_login_attempts
locked_until
created_at
updated_at
```

The last two login-security fields (`failed_login_attempts` and `locked_until`) are part of release hardening and must exist in new installations or be added through defensive migration.

10.3.2.4 cases

A case has a `case_uid`, associated user/examiner, name, description, status, storage directory, and timestamps. The `case_uid` provides stable identity independent of the local integer ID.

10.3.2.5 signatures

The table distinguishes:

```
RC
RO
QUESTIONED
```

It also preserves `forensic_role` for the human classification of questioned signatures when applicable.

10.3.2.6 calibrations

Calibration records, among other items:

```
calibration_uid
case_id
model_id
created_by
calibration_mode
reference_type
distance_name
status
artifact_dir
configuration_path
parameters_json
is_selected
timestamps
```

The core modes documented in this manual are:

```
FORENSIC_CASE
SCIENTIFIC_EXTERNAL
```

Any legacy/additional value present in the schema must not be used as an operational protocol without corresponding documentation and validation.

10.3.2.7 calibration_results

May store:

```
eer
eer_threshold
forensic_threshold
target_far
far
frr
```

```
genuine_count
impostor_count
genuine_mean/genuine_std
impostor_mean/impostor_std
bootstrap confidence interval
metric and plot paths
```

The legacy internal name `forensic_threshold` may exist in storage even when scientific presentation uses the more general term **Calibrated threshold**. Historical persistence must not be renamed destructively solely because of interface terminology.

10.3.2.8 `comparison_runs`

Each execution explicitly links:

```
case_id
model_id
calibration_id
created_by
reference_type
distance_name
threshold
status
artifacts/parameters
```

This structure prevents a result from being interpreted without knowing the model and calibration that produced it.

10.3.2.9 Referential integrity

The runtime enables foreign keys. Scientific records used by comparisons or reports have protections against deletions that would destroy traceability.

Examples:

```
calibration in use → do not freely remove
signature in use → do not freely remove
case with historical evidence → protect
```

10.3.2.10 SQLite in WAL mode

The application uses settings appropriate for a transactional desktop, including WAL, `synchronous`, and `busy_timeout` according to the validated runtime. For backing up an open database, use a consistent mechanism such as the SQLite Backup API.

10.3.3 Intellectual property and software identification

SFAAO has a Computer Program Registration issued by the Brazilian National Institute of Industrial Property (INPI):

```
Process: BR512026005360-9
Title: SFAAO: Sistema Forense de Análise de Assinaturas Offline
       Baseado em Deep Learning e Redes Neurais Siamesas
```

```
       com Calibração Estatística Pericial
Creation date: March 27, 2026
Holder: Universidade do Estado de Mato Grosso - UNEMAT
Author: Diógenes Antonio Marques José
Registered language: Python; others
Certificate issuance: July 14, 2026
```

The existence of the software registration must not be confused with possible patent protection of a method. Patent and technical-secret information must follow the guidance of UNEMAT/AGINOV and the corresponding intellectual-property procedure.

10.3.4 Generative Artificial Intelligence Use Statement

The development of **SFAAO - Forensic Offline Signature Analysis System** received exclusive support from **GPT, through ChatGPT, developed by OpenAI**, as a Generative Artificial Intelligence tool. No other generative-AI system was used in the development process described in this manual.

GPT was used as an assistance tool throughout development, contributing to activities such as source-code analysis and review, identification of inconsistencies, discussion and refinement of software architecture, drafting and reviewing algorithms, structuring training, inference, and calibration pipelines, preparation and review of tests, consistency analysis, technical documentation, methodological organization, and support for scientific writing.

Interaction with GPT was iterative and supervised. Suggestions produced by the model were evaluated, tested, corrected, and validated before incorporation into SFAAO. Therefore, GPT use is declared as **computational assistance to development**, and not as a replacement for human authorship, programming, experimental validation, scientific responsibility, or the professional activity of the examiner.

All decisions concerning SFAAO conception, requirements definition, scientific methodology, calibration protocols, architecture, implementation, testing, validation, interpretation of results, and forensic use remain under human responsibility. GPT does not perform forensic handwriting examination, does not determine document authorship, and does not assume responsibility for forensic conclusions produced with the aid of the platform.

GPT and ChatGPT are technologies developed by **OpenAI**. Historical recognition is also given to **Sam Altman, co-founder of OpenAI**, for his participation in the organization’s trajectory and in the development of the contemporary generative-AI ecosystem. This mention is for recognition and transparency and **does not imply authorship, endorsement, certification, institutional partnership, or responsibility of OpenAI or Sam Altman for SFAAO**.

10.3.4.1 Development credits

```
Concept, development, methodology, and scientific responsibility:
Diógenes Antonio Marques José

Generative Artificial Intelligence assistance:
GPT / ChatGPT - OpenAI

Institutional and historical recognition:
OpenAI; Sam Altman
```

The statement above is part of the project’s transparency policy and must be preserved in official versions of this manual.

Chapter 11

Glossary, checklists and closing remarks

11.1 Glossary

ADMIN — governance role.

EXAMINER — forensic operational role.

Backbone — feature extractor.

Embedding — representative vector.

Pair — pair of samples.

Triplet — anchor, positive, negative.

RC — signatures collected directly from the signatory reproducing the pattern used in contracts; they are known references and do not originate from the questioned contract.

RO — signatures collected directly from the signatory reproducing the pattern used in official documents; they are used as official references for calibration and comparison.

QUESTIONED — any signature present in the contract under examination; all remain questioned even after receiving human forensic classification.

QUESTIONED_COMPATIBLE — QUESTIONED signature classified by the forensic handwriting examiner as compatible with the signatory's pattern; it remains QUESTIONED.

QUESTIONED_IMPOSTOR — human forensic classification used as a non-match in the RC protocol.

FORENSIC_CASE — calibration linked to the case.

SCIENTIFIC_EXTERNAL — independent calibration using external impostors; the case continues to contain RC, RO, and QUESTIONED, but QUESTIONED signatures are excluded from threshold estimation and used only in the subsequent evaluation/inference stage.

FAR — False Acceptance Rate; proportion of non-matches incorrectly accepted. The inequality depends on the metric: cosine uses a high score; Euclidean uses a low distance.

FRR — False Rejection Rate; proportion of matches incorrectly rejected.

EER — Equal Error Rate; rate associated with the point at which FAR and FRR approach each other in the evaluated protocol.

τ_EER — threshold corresponding to the estimated EER.

τ_operational — threshold actually applied; uses metric-dependent orientation, as described in Sections 39 and 40.

Cosine Similarity — angular similarity; a higher value indicates greater compatibility in SFAAO.

Euclidean Distance — geometric distance; a lower value indicates greater compatibility in SFAAO.

DatasetProfile — structural description of a dataset.

DatasetProvider — standardized access to the dataset.

PairBuilder — builder of pair populations.

ComparisonRun — identifiable comparison execution.

Calibration UID — calibration identifier.

Selection hash — SHA-256 of the external selection.

Provenance — verifiable origin of data.

Traceability — reconstruction of the chain that produced a result.

Reproducibility — repetition of the experiment under the same conditions.

WAL — SQLite Write-Ahead Logging.

11.1.1 Glossary of training components

The formal definitions, purpose, time of use, and concrete application of the terms below are consolidated in Section 23.1. This glossary is a quick reference and does not replace the conceptual framework.

PAIR — training mode based on pairs of images labeled as similar or different.

TRIPLET — training mode based on anchor, positive, and negative.

CLASSIFICATION — supervised mode based on classes/identities provided by the dataset.

Optimizer — algorithm responsible for updating the network's trainable parameters.

Adam — adaptive optimizer.

AdamW — Adam variant with decoupled weight decay.

Scheduler — mechanism that changes the learning rate during training.

StepLR — scheduler based on periodic reductions.

Cosine scheduler — scheduler with a cosine curve.

Plateau scheduler — reduces the learning rate when the metric being monitored stabilizes.

Pretrained — backbone initialization with previously learned weights.

Freeze backbone — freezing the parameters of the feature extractor.

Batch size — number of samples processed per optimizer update.

Learning rate — magnitude of parameter updates.

Weight decay — regularization applied during optimization.

Positive ratio — planned proportion of positive pairs.

Forgery negative ratio — planned proportion of negatives constructed from forgeries.

Best state dict — set of weights corresponding to the best state identified during training.

Score distribution — statistical distribution of genuine and impostor scores.

11.2 Operational checklist

```
[ ] correct user
[ ] correct case
[ ] correct examiner
[ ] correct model
[ ] active model
[ ] RC checked
[ ] RO checked
[ ] questioned signatures checked
[ ] forensic roles reviewed
[ ] calibration selected
[ ] threshold checked
[ ] RC comparison executed
```

```
[ ] RO comparison executed
[ ] results reviewed
[ ] report generated
[ ] appendices checked
```

11.3 Release checklist

✓ Linux DEB/RPM embedded images, manifests, and SHA-256 hashes match.

✓ Final DEB/RPM files copied into the distribution without rebuilding.

✓ Validated Windows 2.4.0 installer copied into the distribution.

✓ `06_CHECKSUMS/SHA256SUMS`: all three installers verified `OK`.

✓ SFAAO 2.4.0 functional validation confirmed on Windows 11, Linux Mint (DEB/Podman), and Red Hat (RPM), including DOCX report generation.

✓ Native RPM installation and operation on Red Hat validated per the technical owner; archive the exact OS version and test report in the release dossier.

✓ Updated operational screenshots are included and reviewed; the earlier comparison screen is retained only in the source-image archive.

☐ Editorial review and final customer-material inventory: pending.

☐ Article's final DOI/volume/pages metadata: update once published.

11.3.1 Validation status of the Manual and Release 2.4.0

The release contains SHA-256-verified DEB, RPM, and Windows installers. The technical owner confirmed functional validation on Windows 11, Linux Mint, and Red Hat, including DOCX report generation and RPM installation on Red Hat. This manual records the article's acceptance without implying formal publication. The 23 current operational screenshots are included in this edition; the previous comparison screen is archived separately.

11.4 Conclusion

SFAAO 2.4.0 integrates:

```
CASE MANAGEMENT
+
DEEP LEARNING
+
SIAMESE NETWORKS
+
EMBEDDINGS
+
DATASET PROFILES
+
PAIR / TRIPLET / CLASSIFICATION
+
CONFIGURABLE TRAINING
+
EVALUATION
+
INFERENCE
```

```
+
DUAL CALIBRATION
+
COMPARISON
+
TRACEABILITY
+
AUDITING
+
SECURITY
+
FORENSIC DOCUMENTATION
```

Its purpose is to transform a biometric comparison into a structured, traceable, and technically auditable computational process.

Each result must be interpreted together with the model, data, calibration, threshold, execution, and case context. Final interpretation and the forensic handwriting conclusion remain the responsibility of the qualified professional.

Chapter 12

Bibliographic references and citation

12.1 Scientific and technical references

The references below support the main scientific concepts used in this manual: Deep Learning, Siamese neural networks, contrastive loss, ResNet architectures, biometric metrics, automatic signature verification, signature datasets, and dataset shift.

> **Editorial status of reference [16]:** the authors have confirmed that the SFAAO paper was **accepted for publication in *IEEE Access*** in 2026. Its DOI, volume, issue, pages, and final publication date have not been provided. Cite it as an **accepted paper**, not as a published article, until complete publisher metadata is available.

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015. doi: 10.1038/nature14539.

[2] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a Siamese time delay neural network," in *Advances in Neural Information Processing Systems (NIPS)*, 1994, pp. 737–744. doi: 10.5555/2987189.2987282.

[3] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality Reduction by Learning an Invariant Mapping," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, New York, NY, USA, 2006, pp. 1735–1742. doi: 10.1109/CVPR.2006.100.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[5] A. K. Jain, P. Flynn, and A. A. Ross, *Handbook of Biometrics*. Springer, 2008. doi: 10.1007/978-0-387-71041-9.

[6] L. Prechelt, "Early stopping — but when?" in *Neural Networks: Tricks of the Trade*. Springer, 1998, pp. 55–69. doi: 10.1007/3-540-49430-8_3.

[7] R. Plamondon and G. Lorette, "Automatic signature verification and writer identification — The state of the art," *Pattern Recognition*, vol. 22, no. 2, pp. 107–131, 1989. doi: 10.1016/0031-3203(89)90059-9.

[8] M. A. Ferrer, M. Diaz-Cabrera, and A. Morales, "Static signature synthesis: A neuromotor inspired approach for biometrics," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 3, pp. 667–680, Mar. 2015. doi: 10.1109/TPAMI.2014.2343981.

[9] University of Las Palmas de Gran Canaria, "GPDS: Grupo de Procesado Digital de Señales," 2025. Online resource. Accessed Feb. 25, 2026.

[10] S. Jain, M. Khanna, and A. Singh, "Comparison among different CNN architectures for signature forgery detection using Siamese neural network," in *Proceedings of the International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, 2021, pp. 481–486. doi: 10.1109/ICCCIS51004.2021.9397114.

[11] M. Aleem et al., "Comparative analysis of deep learning models: CNN, MobileNetV2, and ResNet50 for offline signature verification," *Letters in High Energy Physics*, vol. 2024, pp. 1686–1694, 2024.

[12] Y. A. C. Santos, L. C. Rêgo, and R. Ospina, "Online handwritten signature verification via network analysis," *Physica A: Statistical Mechanics and its Applications*, vol. 600, p. 127582, 2022. doi: 10.1016/j.physa.2022.127582.

[13] L. Zhang, Y. Guo, Y. Ding, and H. Sato, “1-D CNN-based online signature verification with federated learning,” *arXiv preprint* arXiv:2406.06597, 2024.

[14] X. Zhao, L. Zheng, P. Yuan, and Y. Zheng, “Fusing deep and hand-crafted features by deep canonically correlated contractive autoencoder for offline signature verification,” *Pattern Recognition*, vol. 168, p. 111834, 2025. doi: 10.1016/j.patcog.2025.111834.

[15] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, “A unifying view on dataset shift in classification,” *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012. doi: 10.1016/j.patcog.2011.06.019.

[16] D. A. M. José, L. de Medeiros Marques, A. Bueno, and R. B. Z. da Silva, “Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification,” *IEEE Access*, accepted for publication, 2026. Manuscript ID Access-2026-35145. Final DOI and bibliographic metadata pending.

[17] PyTorch/TorchVision, “resnet50 — Torchvision documentation.” Official documentation. Available: https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet50.html. Accessed Sep. 8, 2026.

[18] Microsoft, “ProgramData,” Windows Hardware / Shell Setup documentation. Available: https://learn.microsoft.com/windows-hardware/customize/desktop/unattend/microsoft-windows-shell-setup-folderlocations-programdata. Accessed Sep. 8, 2026.

[19] Microsoft, “mklink,” Windows Commands. Available: https://learn.microsoft.com/windows-server/administration/windows-commands/mklink. Accessed Sep. 8, 2026.

[20] Debian Project, “Debian Reference — Chapter 2: Debian package management.” Available: https://www.debian.org/doc/manuals/debian-reference/ch02.en. Accessed Sep. 8, 2026.

[21] Red Hat, “Managing software with the DNF tool — Installing RHEL 9 content.” Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/managing_software_with_the_dnf_tool/assembly_installing-rhel-9-content_managing-software-with-the-dnf-tool. Accessed Sep. 8, 2026.

[22] Podman, “podman-run — volume options and SELinux labeling.” Available: https://docs.podman.io/en/latest/markdown/podman-run.1.html. Accessed Sep. 8, 2026.

[23] PyArmor Project, “PyArmor 9.2 Documentation.” Available: https://pyarmor.readthedocs.io/en/stable/. Accessed Sep. 8, 2026.

12.2 Relationship between manual concepts and references

For ease of technical reading, the main relationships are:

SFAAO topic	Recommended references
Deep Learning	[1]
Siamese neural networks	[2]
Contrastive Loss / metric learning	[3]
ResNet	[4]
FAR, FRR, EER, and biometrics	[5]
Early stopping	[6]
Fundamentals of automatic signature verification	[7]
GPDS and signature synthesis	[8], [9]
Siamese CNN for offline signatures	[10]
Comparison of CNN, MobileNet, and ResNet	[11]
Alternative signature-verification methods	[12]–[14]
Dataset shift	[15]

SFAAO topic	Recommended references
Independent calibration with external impostors, separation among training/calibration/inference, RC/RO forensic study, and methodology directly related to SFAAO	[16]

12.2.1 Recommended citations throughout the manual

When converting this manual into formal academic or technical documentation, it is recommended that citations be inserted in the following sections:

- **Deep Learning and embeddings:** [1];
- **Siamese networks:** [2];
- **Contrastive Loss:** [3];
- **ResNet18/34/50:** [4];
- **FAR, FRR, and EER:** [5];
- **Early stopping:** [6];
- **Offline signature verification:** [7], [10], [11], [14];
- **GPDS / synthetic signature:** [8], [9];
- **Dataset shift:** [15];
- **Independent External-Impostor Calibration:** [16];
- **Separation among training, calibration, and forensic inference:** [16];
- **Exclusion of questioned signatures from scientific calibration:** [16];
- **calibration with 5 references, 10 genuine pairs, and an external impostor population:** [16].

12.3 How to cite SFAAO

Until the software has its own definitive bibliographic publication, there are two complementary forms of reference.

12.3.1 Software reference

```
JOSÉ, Diógenes Antonio Marques. SFAAO – Sistema Forense de Análise de
Assinaturas Offline. Version 2.4.0. Universidade do Estado de Mato Grosso
(UNEMAT), 2026. Software.
```

12.3.2 Scientific reference to the methodology

> JOSÉ, D. A. M.; MARQUES, L. M.; BUENO, A.; SILVA, R. B. Z. *Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification*. IEEE Access, **accepted for publication**, 2026. Manuscript Access-2026-35145. Final DOI and citation metadata pending.

Once formally published, update this citation with the DOI, volume, pages, and other metadata supplied by the publisher without altering the underlying experimental claims.